Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

ОТЧЁТ

по лабораторной работе № 3

«Мир Вампуса» — агент, основанный на знаниях

по дисциплине «Объектно-ориентированное программирование на языке Java»

Выполнил
студент гр. 5130902/40202 _____ А. А. Минязов

Проверил
старший преподаватель _____ Д. С. Хасанов

Санкт-Петербург
2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
1.1 Цель работы 3
1.2 Агент, основанный на знаниях 4
2 Описание модели 5
2.1 Пещера и правила 5
2.2 Случайная пещера 5
2.3 База знаний 6
2.4 Логический вывод 6
2.5 Пример из задания 7
2.6 Стратегия охотника 8
2.7 Результаты и ограничение логики 8
3 Проектирование 10
3.1 Диаграмма прецедентов 10
3.2 Диаграммы деятельности 11
3.3 Диаграммы классов 14
3.4 Диаграмма взаимодействия 16
3.5 Диаграммы состояний 16
3.6 Диаграмма компонентов 18
4 Реализация 19
4.1 Средства разработки 19
4.2 Структура программы 19
4.3 Применение принципов ООП 19
4.4 Оконное приложение 20
4.5 Защита от некорректного ввода 21
4.6 Тестирование 21
5 Демонстрация визуализации 23
Заключение 29
Приложение А. Текст программы 30

1 Постановка задачи

Мир Вампуса — это пещера, состоящая из комнат, — поле N × N. В пещере находятся Вампус, охотник, ямы и золото; распределение объектов выбирается случайным образом. Вампус — животное, которое ест всё в своей комнате; он сторожит золото, лежащее в соседней комнате. Охотник ищет золото; у него одна стрела, которая летит по прямой и может убить Вампуса. Чтобы взять золото, охотник должен убить Вампуса. В пещере есть ямы: охотник может упасть в яму и погибнуть.

Около ямы охотник чувствует ветерок, около Вампуса — запах, около золота — блеск. Датчики охотника: запаха (Stench), ветра (Breeze), блеска (Glitter), удара (Bump) и звука — крика поражённого Вампуса (Scream). Приводы охотника: движение вперёд, поворот на 90 градусов, выстрел вперёд (одна стрела).

Охотник создаёт базу знаний, которая содержит набор предложений. Агент знает правила, действующие в среде, и свои координаты, но не знает конфигурацию среды. На каждом шаге агент выполняет измерение (Stench, Breeze, Glitter, Bump, Scream), делает логический вывод, объединяющий знания, полученные на разных этапах, и принимает решение. Программа должна строить представления агента. Поле может быть разного размера; количество ям увеличивается пропорционально числу комнат.

1.1 Цель работы

Цель работы — разработать на языке Java программу, в которой агент (охотник), не видя карты, по показаниям датчиков строит базу знаний, логически выводит, какие комнаты безопасны, и на её основе действует в пещере; освоить на её примере принципы объектно-ориентированного программирования и основы логического вывода в пропозициональной логике.

1.2 Агент, основанный на знаниях

Агент, основанный на знаниях, хранит знания о мире в виде предложений формального языка и действует по циклу «восприятие → вывод → действие». Используемые термины:

- **база знаний** (KB) — множество предложений, которые агент считает истинными: правила мира и результаты измерений;
- **TELL** — добавить предложение в базу; **ASK** — спросить, следует ли из базы некоторое утверждение;
- **логическое следование** $KB \models \alpha$ — утверждение α истинно при любой расстановке объектов, согласной с базой знаний;
- **предложение** в программе — дизъюнкция литералов, например $P(3,2) \vee P(4,3)$ («яма в 3,2 или в 4,3»); база знаний — их конъюнкция (конъюнктивная нормальная форма).

Проверка $KB \models \alpha$ сводится к выполнимости: α следует из базы тогда и только тогда, когда формула $KB \wedge \neg\alpha$ невыполнима. Выполнимость проверяется алгоритмом DPLL.

2 Описание модели

2.1 Пещера и правила

Комнаты обозначаются так же, как в задании: (строка, столбец), комната (1,1) — левый верхний угол. Правила мира приведены в таблице 1.

Таблица 1 — Правила мира

Что	Как
Пещера	N × N комнат, N от 4 до 12
Старт	левый нижний угол (N,1); охотник смотрит вправо
Ямы	доля от числа комнат, по умолчанию 20 %; вошедший в яму погибает
Вампус	один, неподвижен; вошедший к живому Вампусу погибает
Золото	одно, в комнате, соседней с Вампусом; взять можно только после убийства Вампуса
Запах	Вампус (живой или мёртвый) в этой или соседней комнате
Ветерок	яма в соседней комнате
Блеск	золото (пока не взято) в этой или соседней комнате
Удар / крик	шаг в стену / стрела попала в Вампуса — только в ход действия
Действия	шаг вперёд, поворот налево и направо, выстрел, взять золото, выбраться из пещеры (только на старте)
Счёт	−1 за действие, −10 за выстрел, −1000 за гибель, +1000 за выход с золотом

В задании не указано, куда смотрит охотник в начале; выбрано направление «вправо», как в классической постановке задачи. При нём выбор ходов совпадает с примером из задания (4,1 → 4,2 → 3,1). Действия «взять золото» и «выбраться» в списке приводов задания нет, но без них игру нельзя закончить.

2.2 Случайная пещера

Пещера строится по заказу: размер, доля ям в процентах и зерно случайности (seed). Одинаковый seed всегда даёт одинаковую пещеру. Старт и соседние с ним комнаты остаются без ям и Вампуса, иначе часть игр заканчивалась бы на первом шаге. Вампус ставится в случайную комнату вне

старта, золото — в случайную соседнюю с ним комнату, ямы (round(доля · N^2) штук) — в случайные из оставшихся комнат.

2.3 База знаний

Каждый факт — переменная «да/нет»: P(r,c) — в комнате яма, W(r,c) — Вампус, G(r,c) — золото; на пещеру N × N приходится $3N^2$ переменных. С самого начала в базу записаны правила мира:

– Вампус ровно один: W(1,1) ∨ … ∨ W(N,N) и ¬W(a) ∨ ¬W(b) для каждой пары комнат; так же для золота;
– золото рядом с Вампусом: ¬G(c) ∨ W($сосед_1$) ∨ … ∨ W($сосед_k$) для каждой комнаты c;
– Вампус и золото не лежат в яме: ¬W(c) ∨ ¬P(c), ¬G(c) ∨ ¬P(c).

После каждого измерения охотник записывает в базу новые предложения (таблица 2). Правило «ветерок ⇔ яма в одной из соседних комнат» при известном измерении превращается либо в одну дизъюнкцию, либо в отрицания для каждого соседа.

Таблица 2 — Измерения и предложения базы знаний

Измерение	Предложения
ветерок в 4,2	P(3,2) ∨ P(4,3) ∨ P(4,1)
ветерка в 4,1 нет	¬P(3,1), ¬P(4,2)
запах в 3,1	W(3,1) ∨ W(2,1) ∨ W(3,2) ∨ W(4,1)
блеска в 4,1 нет	¬G(4,1), ¬G(3,1), ¬G(4,2)
охотник жив в комнате 4,1	¬P(4,1), ¬W(4,1) (пока Вампус не убит)
выстрел вверх из 4,1, крик	W(3,1) ∨ W(2,1) ∨ W(1,1)
выстрел вправо из 4,1, крика нет	¬W(4,2), ¬W(4,3), ¬W(4,4)

2.4 Логический вывод

Для каждого факта охотник выясняет, может ли он быть истинным и может ли быть ложным при условии, что верны все предложения базы. Если возможно только одно значение, факт доказан: «точно да» (пометка P!, W!, G!)

или «точно нет». Если возможны оба — «может быть» (P?, W?, G?). Комната безопасна (пометка OK), если доказано, что ямы в ней нет и что живого Вампуса в ней нет.

Вопрос «может ли P(3,2) быть истинным» — это проверка выполнимости формулы KB ∧ P(3,2). Её решает алгоритм DPLL: он по очереди задаёт значения переменных, после каждого выводит следствия из предложений, где остался один незаданный литерал (распространение единичных предложений), и при противоречии откатывается. В программе используется DPLL с обучением на конфликтах (CDCL): из противоречия выводится новое предложение — его причина, и откат идёт сразу к решению, которое к ней привело. Простой DPLL с откатом на одно решение назад на случайных пещерах 6 × 6 не заканчивал работу: при каждом противоречии он перебирал ямы, не относящиеся к делу.

Перебирать все 2^(3N²) расстановок не требуется ещё по двум причинам. Найденная DPLL расстановка показывает возможные значения сразу всех фактов, поэтому новый запуск нужен только для фактов, значение которых ещё не встречалось. Доказанный факт остаётся доказанным навсегда, потому что база только растёт, — он запоминается. Игра в пещере 6 × 6 занимает около 15 мс, самое долгое действие в пещере 12 × 12 (около 21 тыс. предложений) — около 140 мс.

2.5 Пример из задания

В пещере из задания Вампус находится в 2,1, золото — в 2,2, ямы — в 1,4, 2,3 и 4,3. Программа повторяет шаги задания (таблица 3).

Таблица 3 — Шаги вывода в примере из задания

Шаг	Измерение	Вывод	Решение
1, комната 4,1	(none, none, none, none, none)	безопасны: 3,1, 4,2	иду в 4,2
2, комната 4,2	(none, Breeze, none, none, none)	яма может быть в 3,2, 4,3	иду в 3,1
3, комната 3,1	(Stench, none, none, none, none)	безопасна 3,2; яма в 4,3; Вампус в 2,1	стреляю вверх

На шаге 3 вывод объединяет знания разных шагов. Ветерка в 3,1 нет — значит, в 3,2 ямы нет, и яма, которую требует ветерок в 4,2, находится в 4,3. Запах в 3,1 означает Вампуса в 2,1 или 3,2, но в 4,2 запаха не было, а 3,2 — её соседка, значит, Вампус в 2,1. В задании в выводе шага 3 указано «Вампус в клетке 3.1»; по рисунку задания и по логике правильно 2,1 — это опечатка. Дальше охотник стреляет, слышит крик, по блеску выводит, что золото в 2,2, берёт его и выходит с золотом; итоговый счёт +968.

2.6 Стратегия охотника

Охотник выбирает цель, когда предыдущее решение выполнено, в таком порядке:

- золото у него — идти к выходу и выбраться;
- Вампус убит и доказано, где золото, — идти за золотом и взять его;
- стрелы нет, а Вампус жив — идти к выходу: золото уже не взять;
- доказано, где Вампус, — встать с ним на одну линию и выстрелить;
- идти в ближайшую непосещённую безопасную комнату;
- безопасных комнат нет и риск разрешён — выстрелить наугад по линии, где может быть Вампус, а если нельзя — шагнуть в комнату, где яма не доказана;
- иначе — идти к выходу без золота.

Охотник ходит только через комнаты, где уже был. Путь строится поиском в ширину; из нескольких целей выбирается та, до которой меньше действий с учётом поворотов. Решение превращается в план — последовательность действий, — которую охотник выполняет, пока план не кончится; придя к цели, он сразу делает следующий вывод.

2.7 Результаты и ограничение логики

Охотник сыграл по 1000 случайных пещер каждого размера с долей ям 20 % (таблица 4).

Таблица 4 — Результаты на случайных пещерах

Пещера	Побед с риском	Гибель с риском	Побед без риска
4 × 4	70 %	5 %	64 %
6 × 6	45 %	6 %	38 %
8 × 8	28 %	7 %	23 %

Без риска охотник не погибает ни разу. При полной карте почти все пещеры проходимы (99, 99 и 96 %), поэтому проигрыши объясняются не ошибкой в выводе. На больших полях ветерок чувствуется почти везде, и измерений не хватает, чтобы доказать безопасность хотя бы одной новой комнаты: из 231 проигрыша без риска на поле 8 × 8 в 227 охотник ушёл, так и не найдя Вампуса. Логический вывод даёт только достоверные заключения; оценивать, какая из недоказанных комнат опаснее, он не умеет — для этого нужен вероятностный вывод, которого задание не требует.

3 Проектирование

Система описана UML-диаграммами шести видов: прецедентов, деятельности, классов, взаимодействия (последовательности), состояний и компонентов. Диаграммы построены в PlantUML.

3.1 Диаграмма прецедентов

Пользователь открывает пример из задания или создаёт случайную пещеру, выполняет игру по шагам или непрерывно, начинает её сначала, скрывает настоящую пещеру, разрешает или запрещает риск и смотрит на знания охотника и журнал (рисунок 1). Второй актёр — охотник: он измеряет, записывает измерения в базу знаний, спрашивает её и выбирает действие.

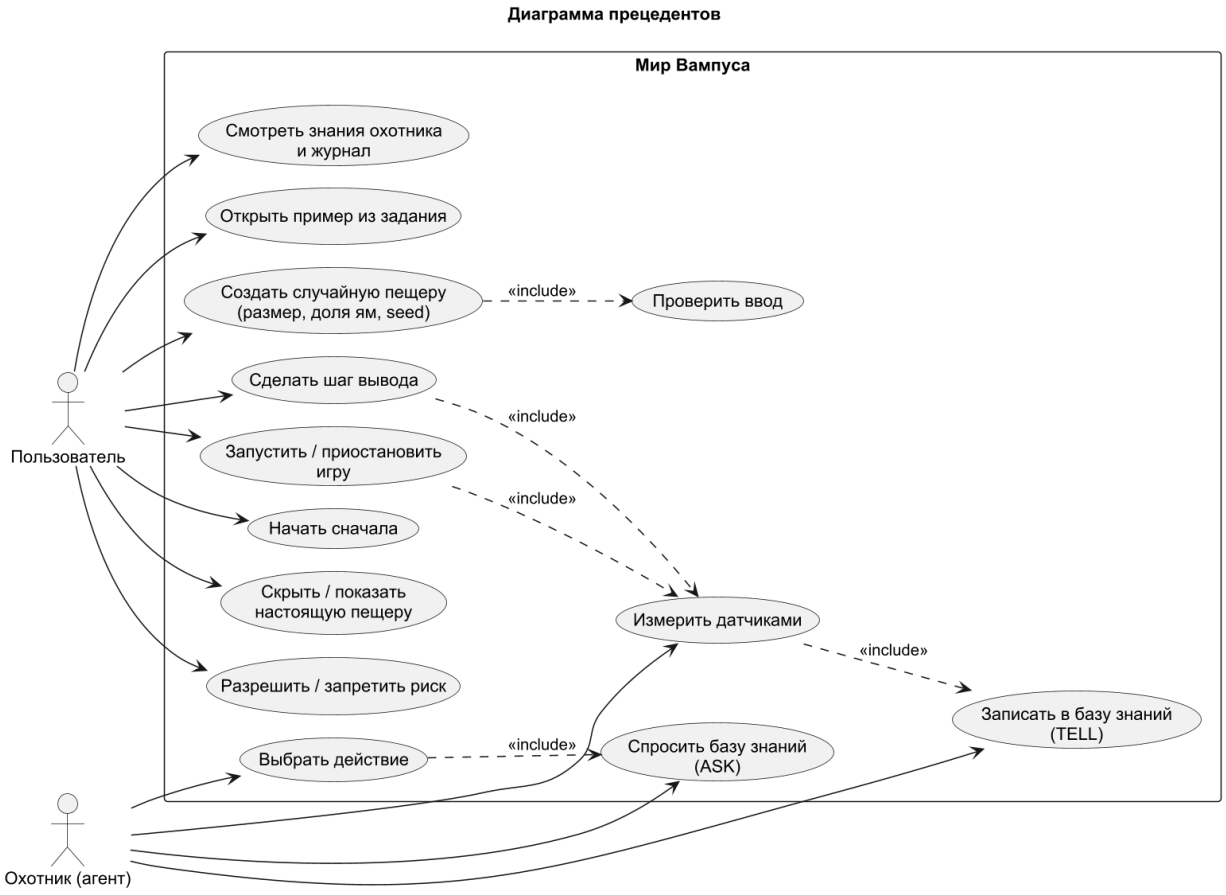


Рисунок 1 — Диаграмма прецедентов

3.2 Диаграммы деятельности

На рисунке 2 показан шаг охотника: измерение, запись в базу знаний и, если прошлое решение выполнено, вывод статусов всех фактов и выбор новой цели.

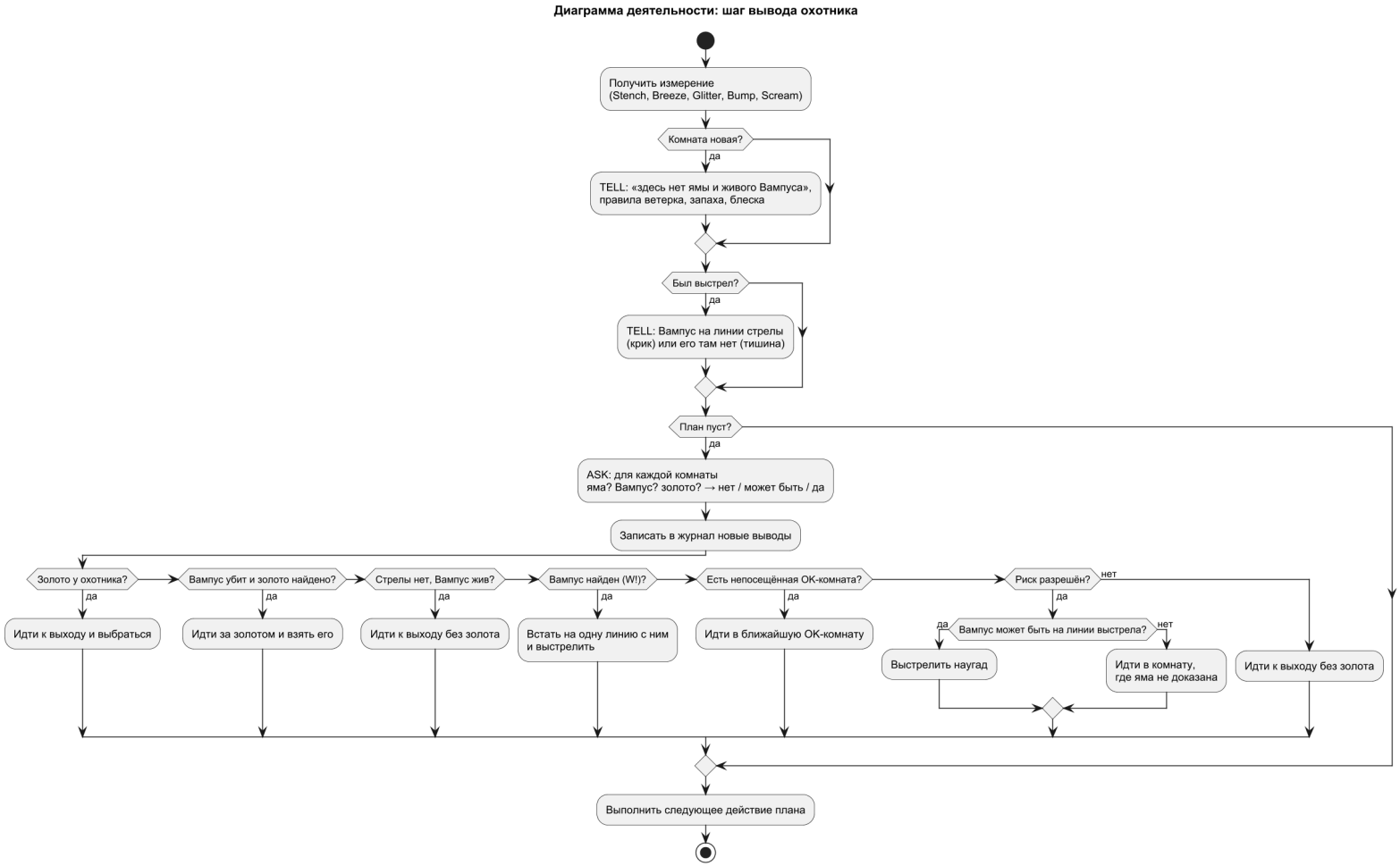


Рисунок 2 — Диаграмма деятельности: шаг вывода охотника

На рисунке 3 показан вывод статуса одного факта: проверка сохранённых результатов, найденных расстановок и, при необходимости, запуск DPLL.

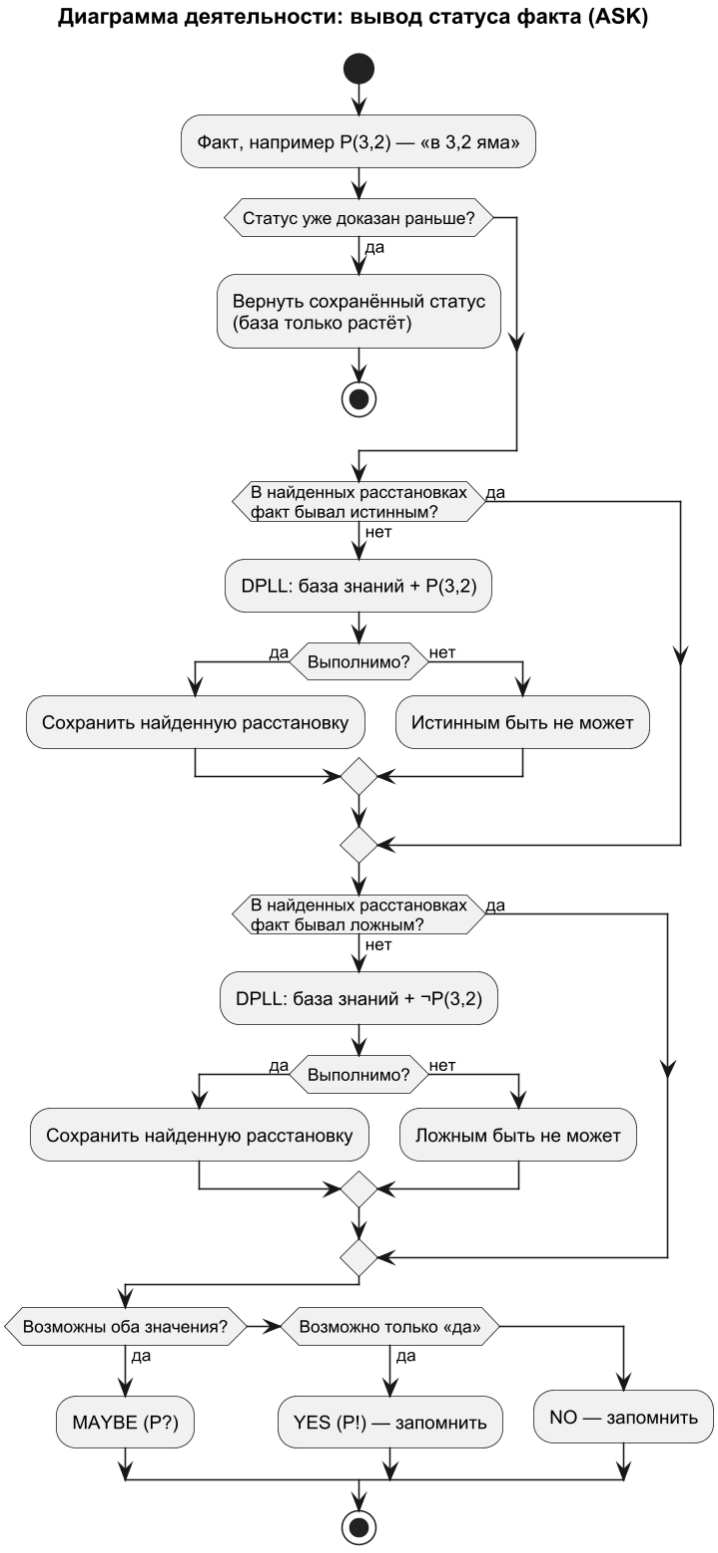


Рисунок 3 — Диаграмма деятельности: вывод статуса факта (ASK)

3.3 Диаграммы классов

Классы мира и логики показаны на рисунке 4. Cave хранит расстановку, CaveGenerator строит её по заказу CaveSpec, World — среда: выполняет действия, выдаёт измерения и считает очки. Fact, Literal и Clause — предложения базы знаний, Rules переводит правила мира и измерения в предложения, KnowledgeBase отвечает на вопросы с помощью Dpll.

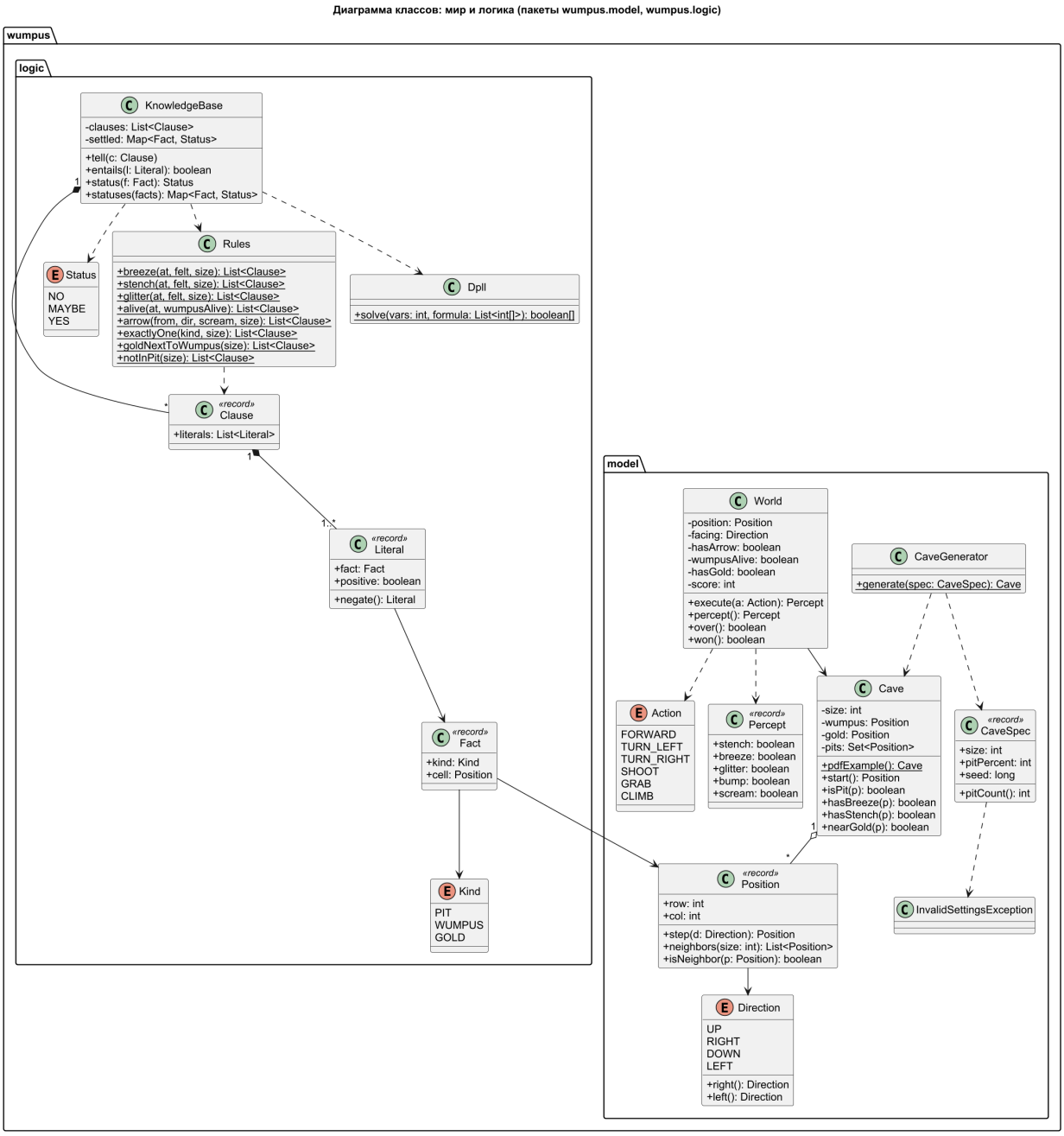


Рисунок 4 — Диаграмма классов: мир и логика

Классы охотника, управления и окна показаны на рисунке 5. Hunter хранит базу знаний, план и журнал (записи Turn), Planner строит пути. Логика кнопок (GameController), пометок на карте (Marks) и формы (CaveForm) не зависит от JavaFX; окно только передаёт действия пользователя контроллеру и рисует состояние.

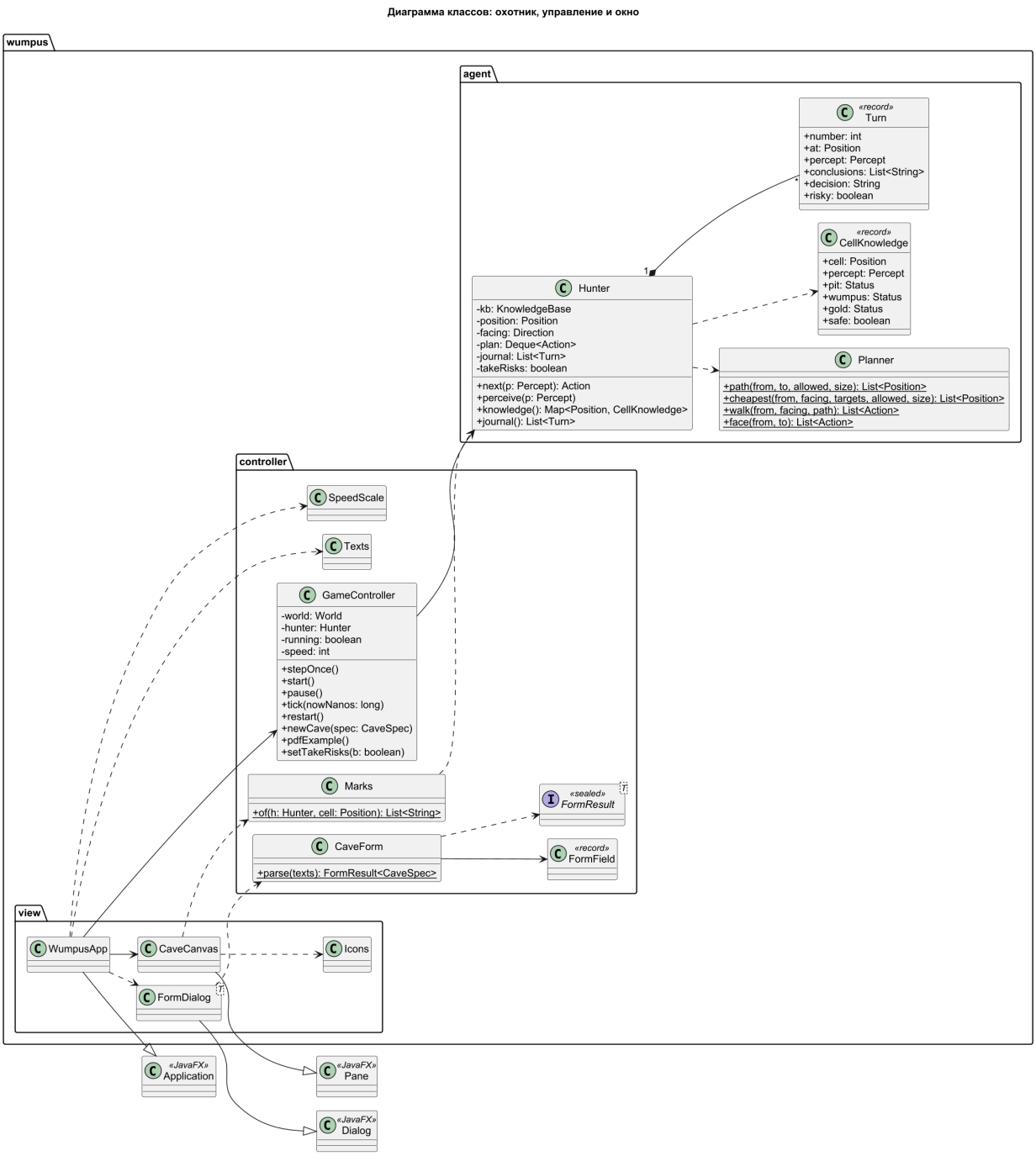


Рисунок 5 — Диаграмма классов: охотник, управление и окно

3.4 Диаграмма взаимодействия

На рисунке 6 показано взаимодействие объектов при нажатии кнопки «Шаг»: охотник отдаёт следующее действие своего плана (поворот, шаг, выстрел), мир выполняет его и возвращает измерение, охотник сразу записывает новое в базу знаний и, если решение выполнено, тут же делает вывод и выбирает новую цель — так строка журнала появляется в момент прихода в комнату.

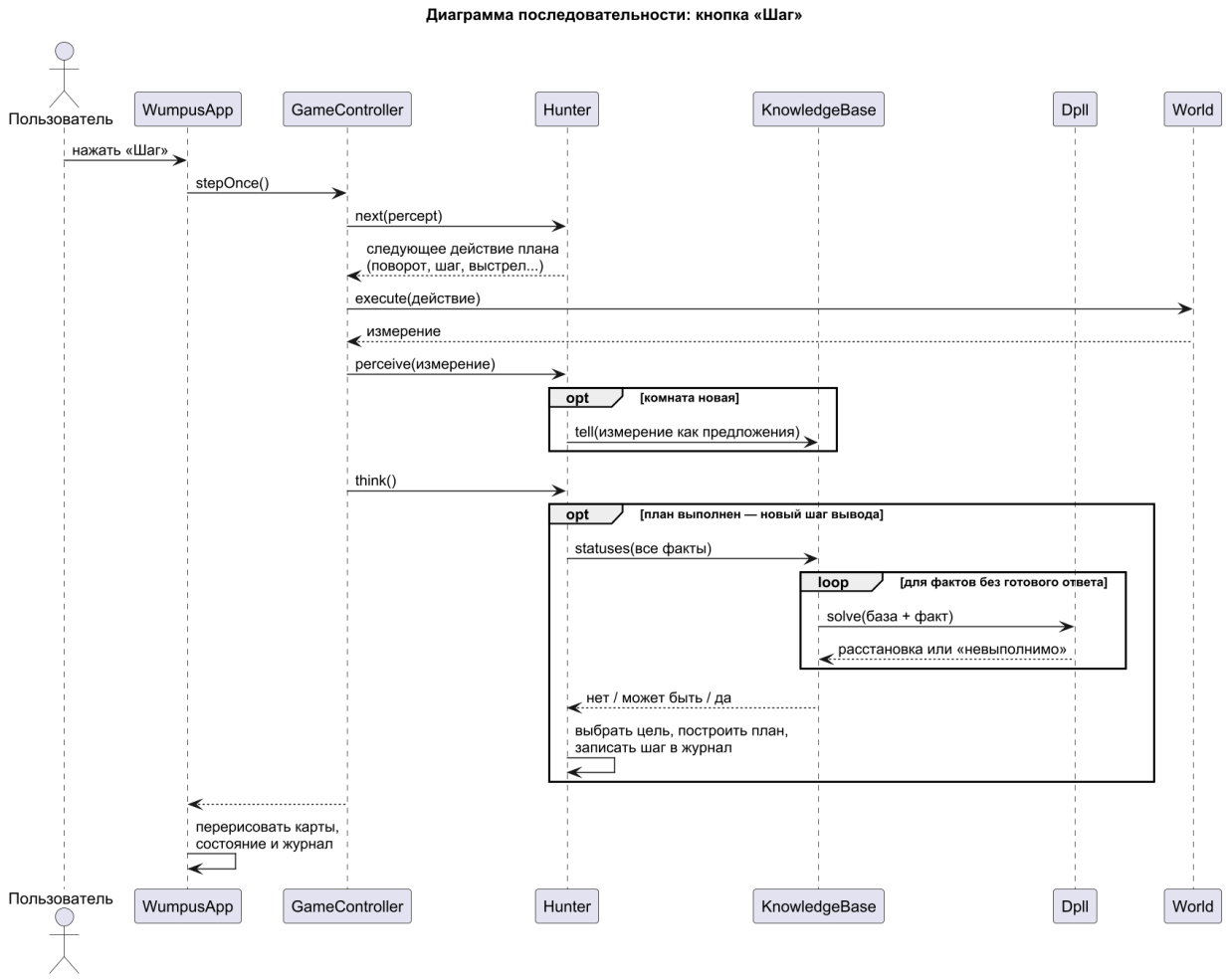


Рисунок 6 — Диаграмма последовательности: кнопка «Шаг»

3.5 Диаграммы состояний

На рисунке 7 показаны состояния игры в окне, на рисунке 8 — смена целей охотника.

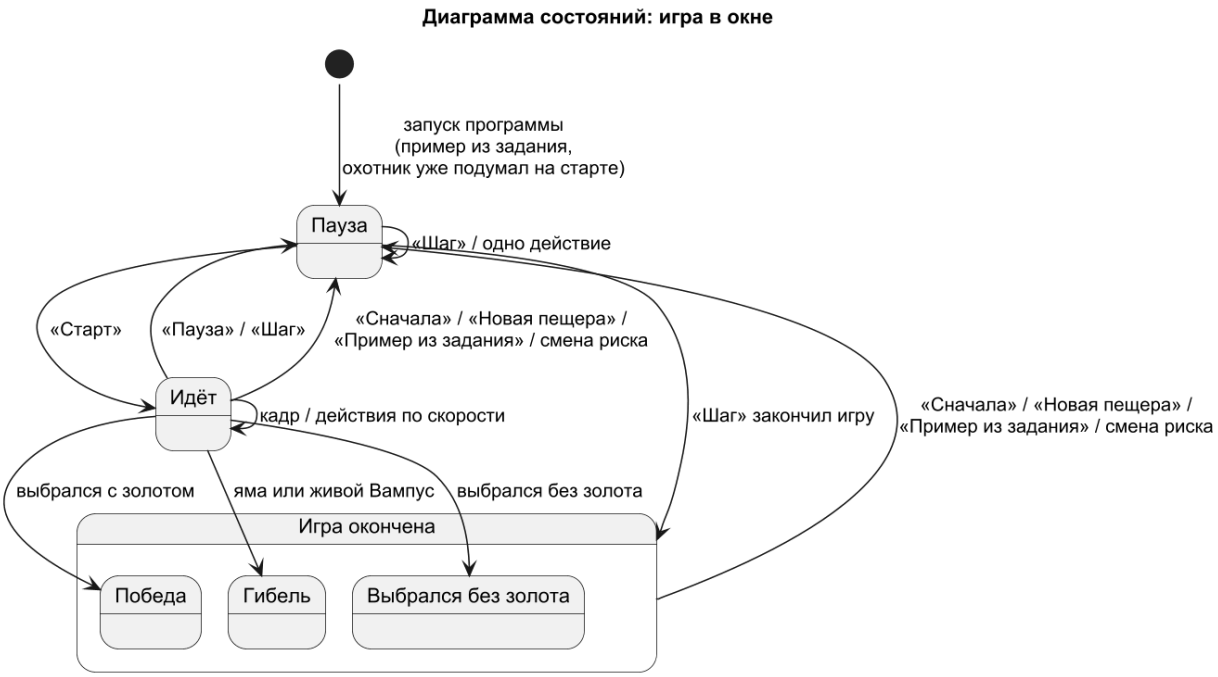


Рисунок 7 — Диаграмма состояний: игра в окне

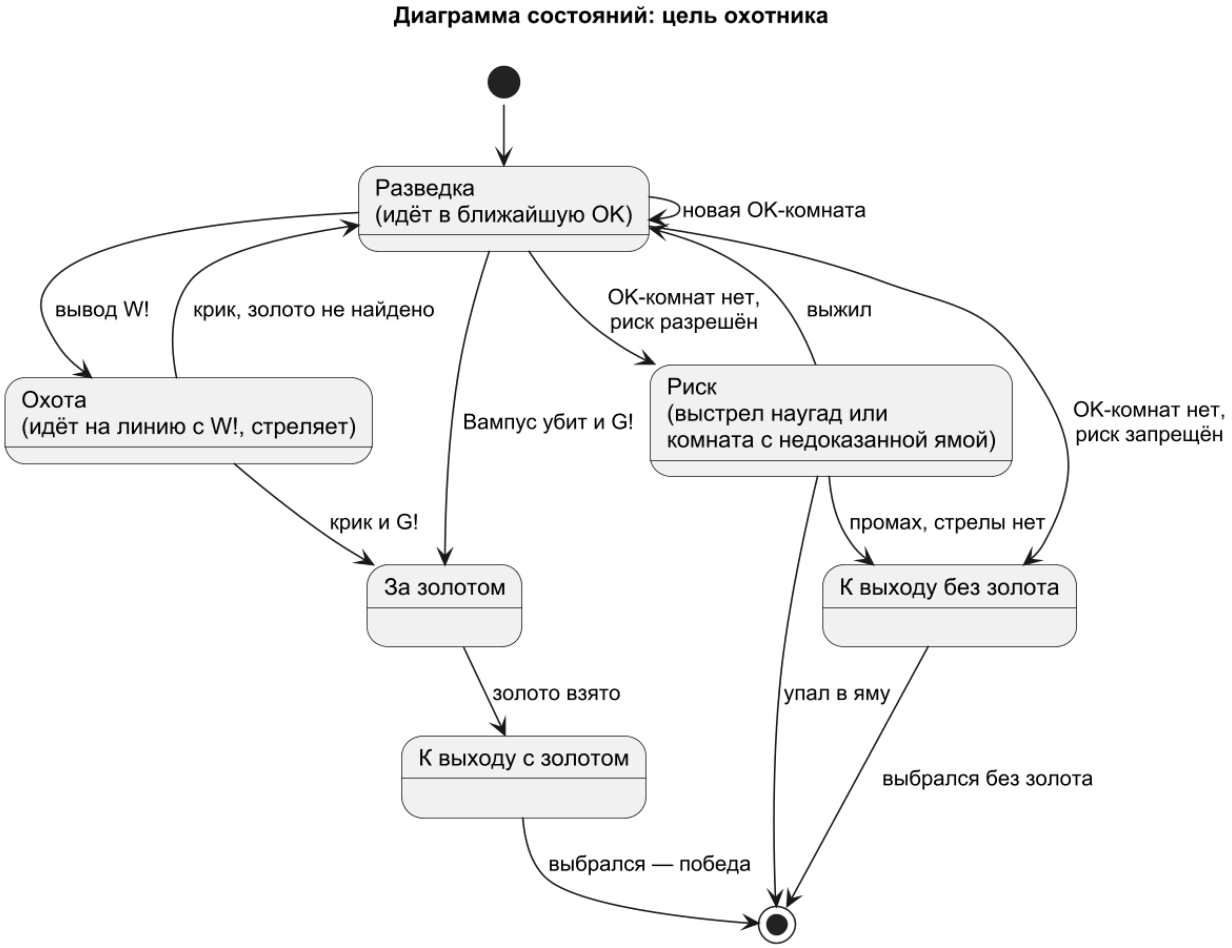


Рисунок 8 — Диаграмма состояний: цель охотника

3.6 Диаграмма компонентов

Программа оформлена как модуль Java wumpus и разделена на пакеты (рисунок 9). Мир не зависит ни от охотника, ни от окна; логика зависит только от мира; окно зависит от библиотеки JavaFX.

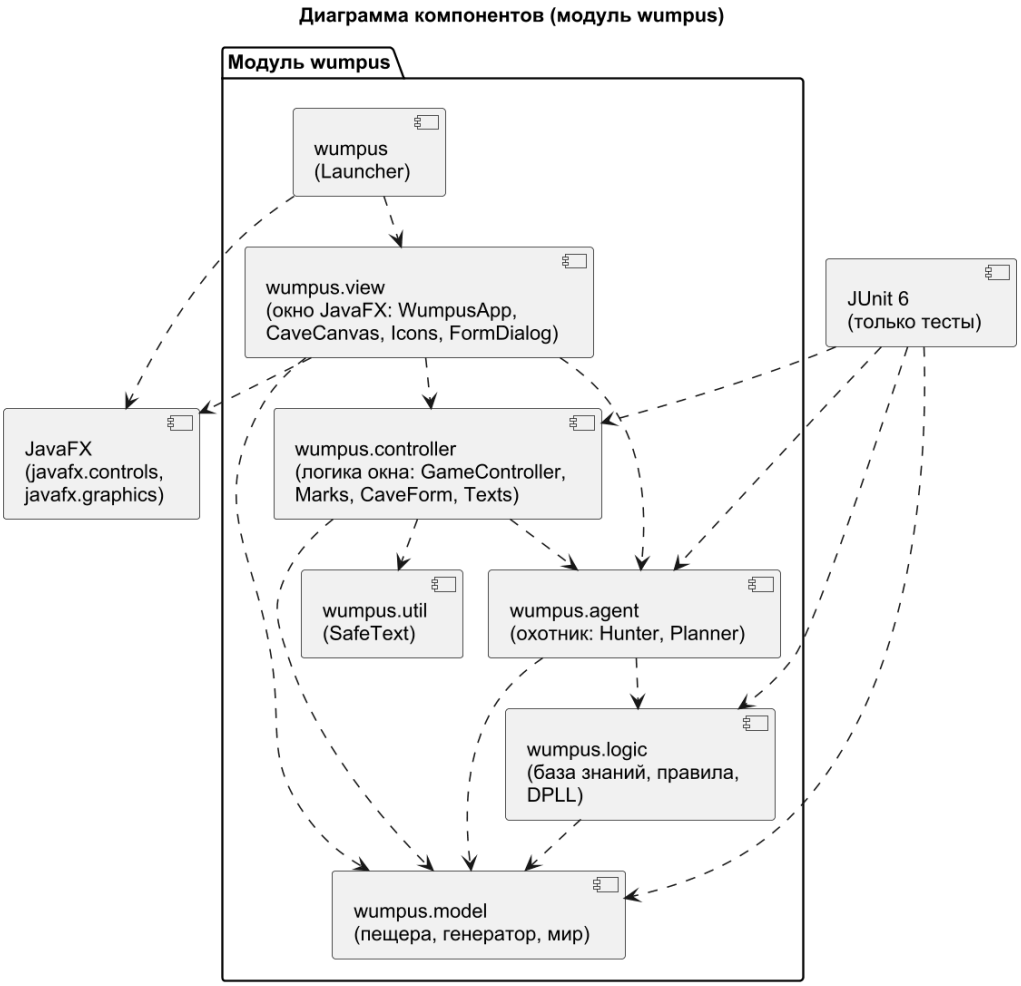


Рисунок 9 — Диаграмма компонентов

4 Реализация

4.1 Средства разработки

– язык Java 25 (JDK Temurin 25);
– библиотека пользовательского интерфейса JavaFX 25.0.4;
– система сборки Maven, среда разработки IntelliJ IDEA;
– библиотека тестирования JUnit 6, система контроля версий Git.

4.2 Структура программы

Таблица 5 — Пакеты программы

Пакет	Назначение
wumpus	Launcher — точка входа
wumpus.model	Position, Direction, Action, Percept — комнаты, направления, действия, измерение; Cave, CaveSpec, CaveGenerator — пещера; World — среда; InvalidSettingsException
wumpus.logic	Kind, Fact, Literal, Clause — предложения; Rules — правила мира; KnowledgeBase — TELL и ASK; Status — нет / может быть / да; Dpll — проверка выполнимости
wumpus.agent	Hunter — охотник; Planner — пути и повороты; CellKnowledge — знания о комнате; Turn — строка журнала
wumpus.controller	GameController (старт, пауза, шаг — одно действие, скорость, новая пещера), Marks (пометки карты), CaveForm, FormField, FormResult, Forms (форма), Texts, SpeedScale
wumpus.view	WumpusApp (главное окно), CaveCanvas (карты), Icons (значки), FormDialog (окно новой пещеры)
wumpus.util	SafeText — безопасный вывод введённого текста в сообщениях

4.3 Применение принципов ООП

Таблица 6 — Принципы ООП в программе

Принцип	Реализация
Инкапсуляция	расстановка пещеры неизменяема и закрыта; состояние игры (позиция, стрела, золото, счёт) меняется только методом World.execute; база знаний хранит предложения в закрытом виде и кодирует их для DPLL сама; охотник не имеет доступа к пещере и получает только измерения

Принцип	Реализация
Абстракция	охотник работает с базой знаний через TELL и ASK, не зная устройства DPLL; факты, литералы и предложения — записи (record) с понятным текстовым видом; результат разбора формы — закрытая (sealed) иерархия FormResult
Наследование	WumpusApp наследует Application, CaveCanvas — Pane, FormDialog — Dialog, InvalidSettingsException — IllegalArgumentException
Полиморфизм	перечисления Direction, Action, Kind со своими методами и обработка действий сопоставлением с образцом (switch); один класс CaveCanvas рисует обе карты в двух режимах; обобщённый FormDialog<T>; JavaFX вызывает переопределённые start и layoutChildren

4.4 Оконное приложение

Главное окно (класс WumpusApp) показывает две карты: слева — настоящую пещеру со значками ям, Вампуса, золота и подсказками датчиков (запах, ветерок, блеск), справа — то, что знает охотник: посещённые комнаты серые, пометки OK, P?, P!, W?, W!, G?, G! — как в таблицах задания. Пометка «может быть» ставится только рядом с комнатой, где почувствовали ветерок, запах или блеск, иначе она стояла бы почти в каждой неизвестной комнате. Охотник нарисован на обеих картах, красная стрелка показывает, куда он смотрит. Справа — состояние игры: номер шага вывода, число действий, счёт, стрела, что известно о Вампусе и золоте. Внизу — журнал в формате задания: строка на каждый шаг вывода — комната, измерение, вывод, решение и действия, которыми оно выполняется; охотник думает сразу, как пришёл в комнату, поэтому строка и пометки на карте появляются в этот момент (таблица 7).

Таблица 7 — Элементы управления

Элемент	Действие
Старт / Пауза	охотник действует сам с выбранной скоростью
Шаг	одно действие охотника: поворот, шаг вперёд, выстрел, взять золото или выбраться
Сначала	та же пещера, охотник снова ничего не знает
Новая пещера…	размер N (4–12), доля ям (0–40 %), seed
Пример из задания	пещера 4 × 4 с рисунка задания
показывать правду	скрыть настоящую пещеру

Элемент	Действие
рисковать	разрешить выстрел наугад и шаг в комнату с недоказанной ямой
Скорость	от 1 до 50 действий в секунду, логарифмическая шкала

Непрерывная игра реализована таймером JavaFX AnimationTimer, который вызывается при каждой перерисовке экрана. Контроллер накапливает прошедшее время, умноженное на скорость, и выполняет столько действий, сколько накопилось; время на паузе не превращается в действия, а после долгого «замерзания» окна учитывается не больше секунды. Окно проверено при размерах от минимального 900 × 620 до полноэкранного и на пещерах до 12 × 12: панель состояния в низком окне прокручивается, в мелких клетках не пишутся координаты.

4.5 Защита от некорректного ввода

Разбор полей окна «Новая пещера» выполняет класс CaveForm. Программа не завершается аварийно ни при каком вводе: пустое поле, буквы, дроби, запись с экспонентой (1e3), NaN, цифры других алфавитов, слишком большие числа и управляющие символы приводят к понятному сообщению с указанием поля; длинный ввод в сообщении обрезается. Кроме формата проверяется смысл: размер от 4 до 12, доля ям от 0 до 40 %. Обо всех ошибках сообщается сразу, неверные поля подсвечиваются, окно не закрывается.

4.6 Тестирование

Программа проверена 171 автоматическим тестами JUnit, которые запускаются в случайном порядке. Главные проверки:

– шаги 1–3 из задания дают ровно те же измерения, выводы и решения;
– DPLL сверяется с полным перебором на 4500 случайных формулах, в том числе на самых трудных (около 4,3 предложения на переменную);

– всё, что охотник считает доказанным, совпадает с настоящей пещерой в 240 случайных играх;
– охотник погибает только после рискованного решения и никогда не идёт в доказанно опасную комнату; без риска не погибает ни разу;
– логика кнопок, таймера, формы и пометок проверяется без открытия окна.

Тесты проверены мутациями: в код вносилась намеренная ошибка (например, «>» заменялось на «>=» или убирался вызов), и тесты должны были её обнаружить. Найденные так пробелы закрыты новыми тестами, а условия, которые не встречаются ни в одной игре, удалены из кода.

5 Демонстрация визуализации

На рисунке 10 показано окно при запуске — пещера из задания: охотник на старте и уже сделал первый вывод: в журнале шаг 1, на его карте соседние комнаты 3,1 и 4,2 помечены OK. Кнопка «Шаг» выполняет одно действие; строка журнала появляется, когда охотник пришёл в новую комнату и подумал. На рисунке 11 охотник дошёл до 3,1: журнал повторяет шаги 1–3 задания вместе с действиями (например, путь из 4,2 в 3,1 — направо, направо, вперёд, направо, вперёд), на карте доказаны яма в 4,3 (P!) и Вампус в 2,1 (W!), комната 3,2 помечена OK; следующее действие — выстрел вверх.

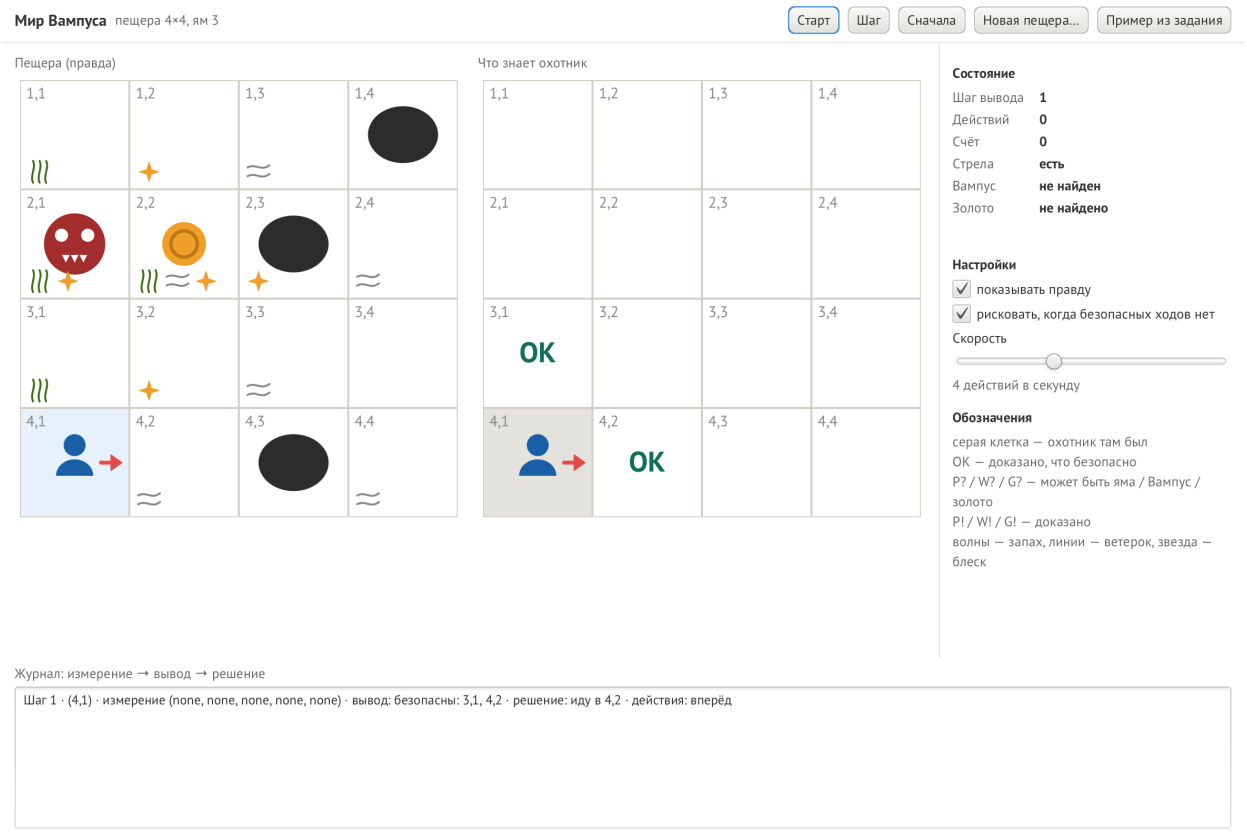


Рисунок 10 — Окно при запуске: пример из задания

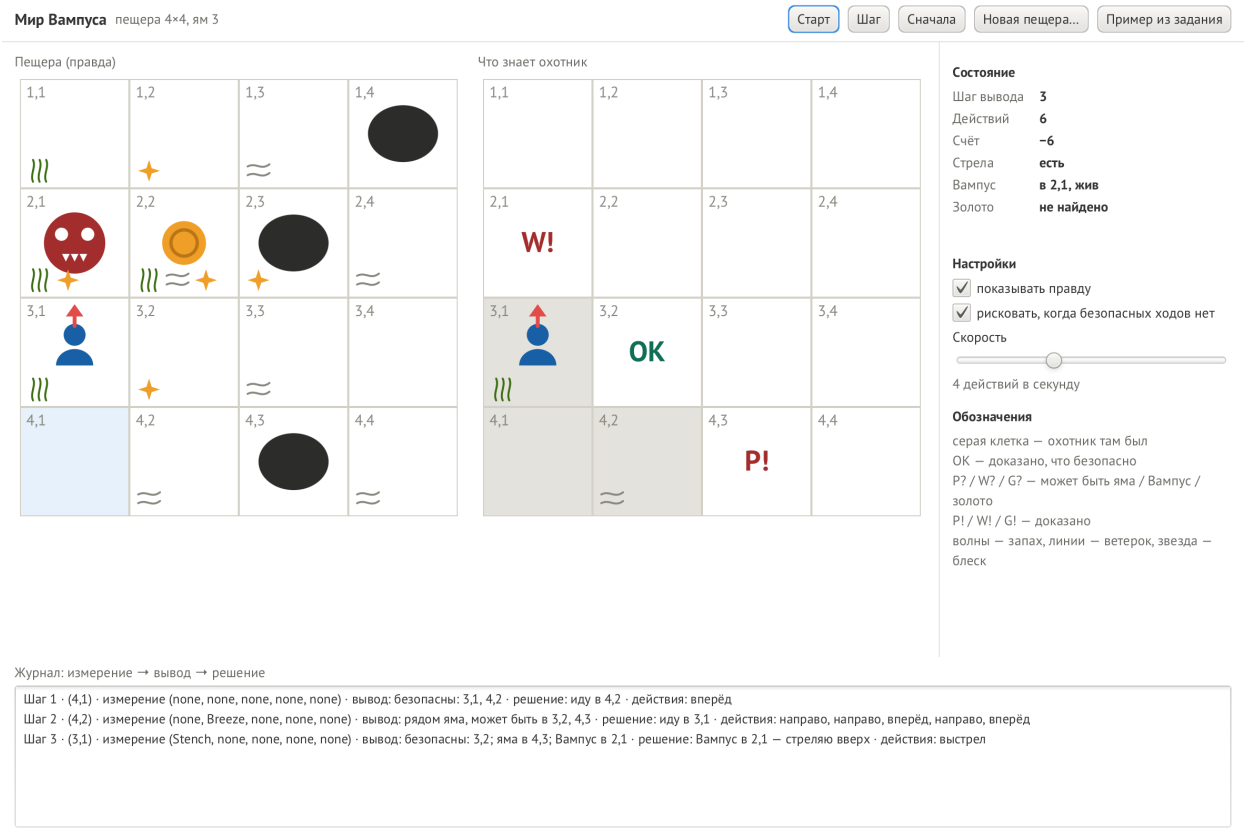


Рисунок 11 — После трёх шагов вывода

На рисунке 12 — та же игра до конца: охотник вывел по блеску, где золото, взял его и выбрался; счёт +968. На рисунке 13 настоящая пещера скрыта: видно только то, что знает охотник, — те же выводы после трёх шагов, что и на рисунке 11.

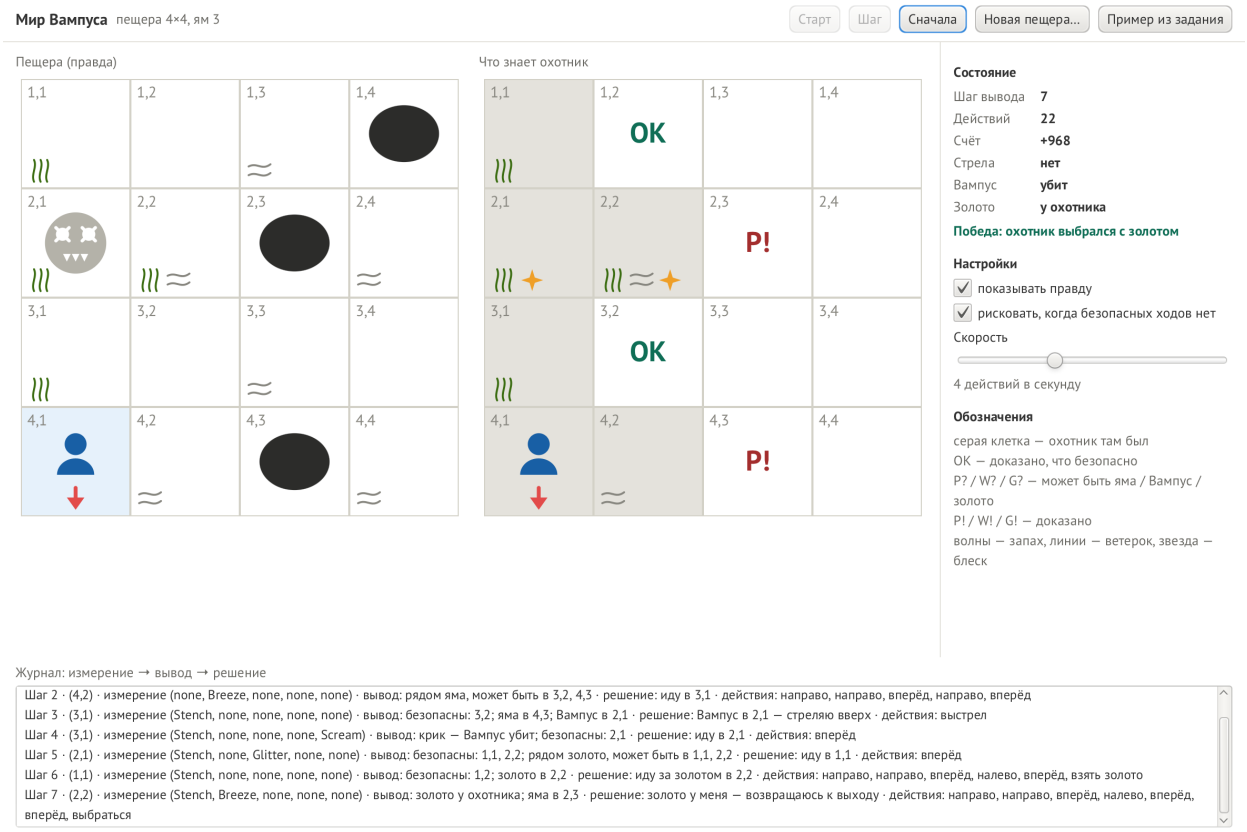


Рисунок 12 — Победа в примере из задания

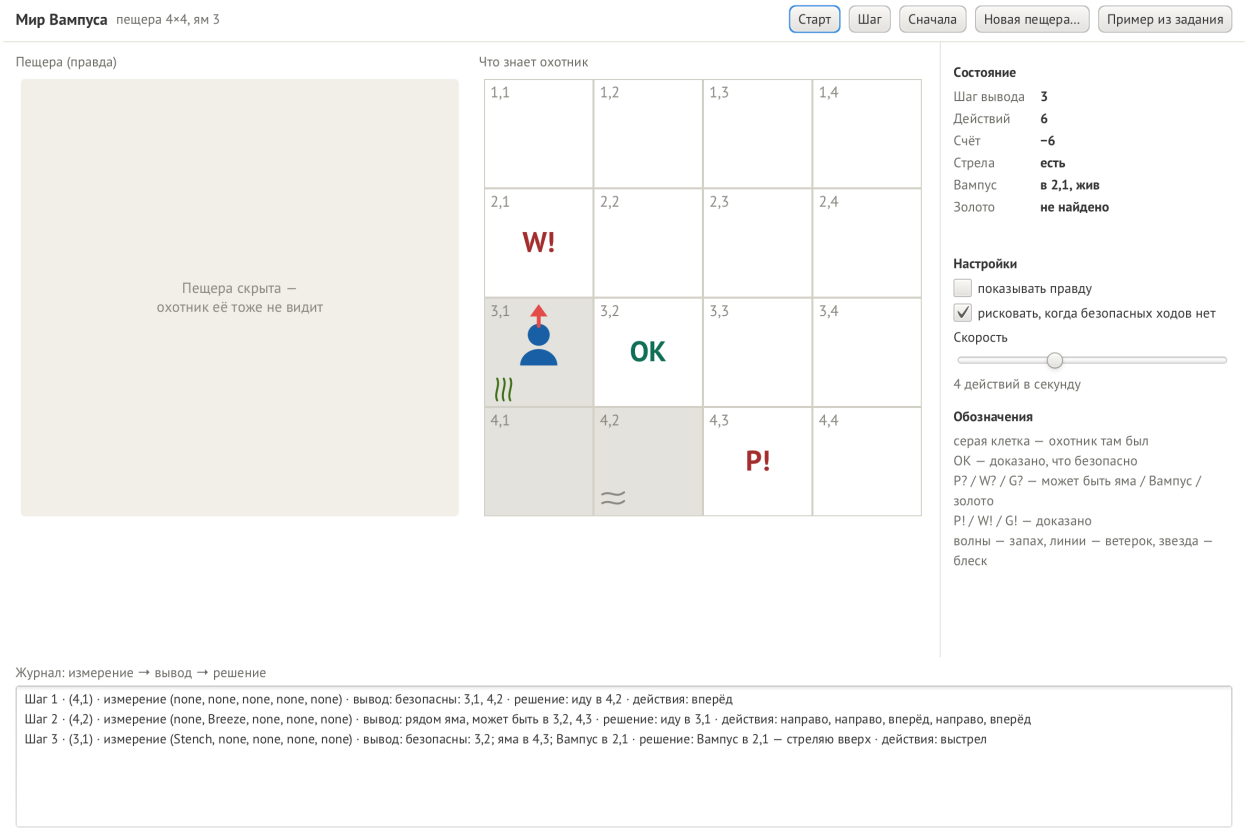


Рисунок 13 — Настоящая пещера скрыта

На рисунке 14 — случайная пещера 10 × 10 с 20 ямами: охотник по блеску и запаху вывел, что Вампус в 9,5, а золото в 10,5, застрелил Вампуса из 9,4, взял золото и выбрался. На рисунке 15 — пещера 6 × 6, где безопасных ходов не осталось: охотник выстрелил наугад и убил Вампуса, затем рискнул и упал в яму 3,2. На рисунке 16 та же пещера пройдена без риска: охотник обошёл все доказанно безопасные комнаты и вышел без золота, не погибнув.

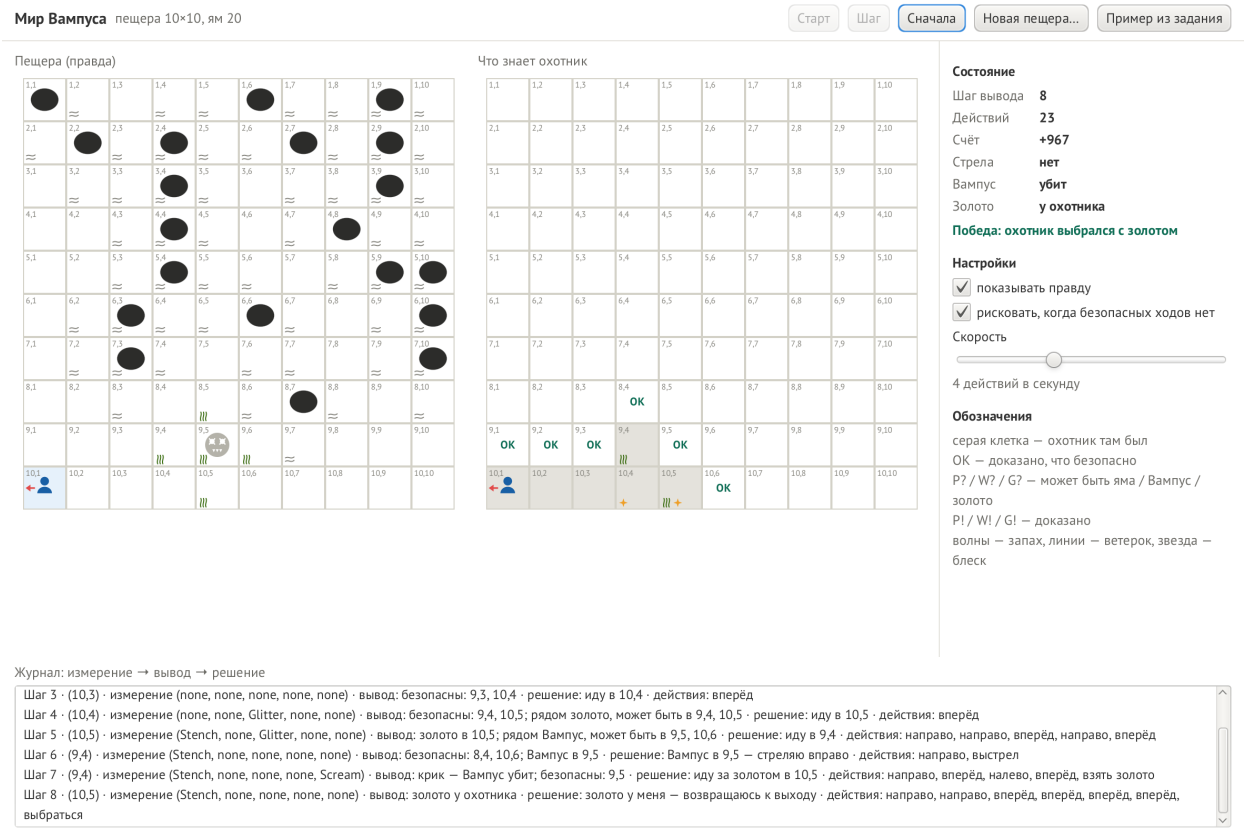


Рисунок 14 — Пещера 10 × 10: победа

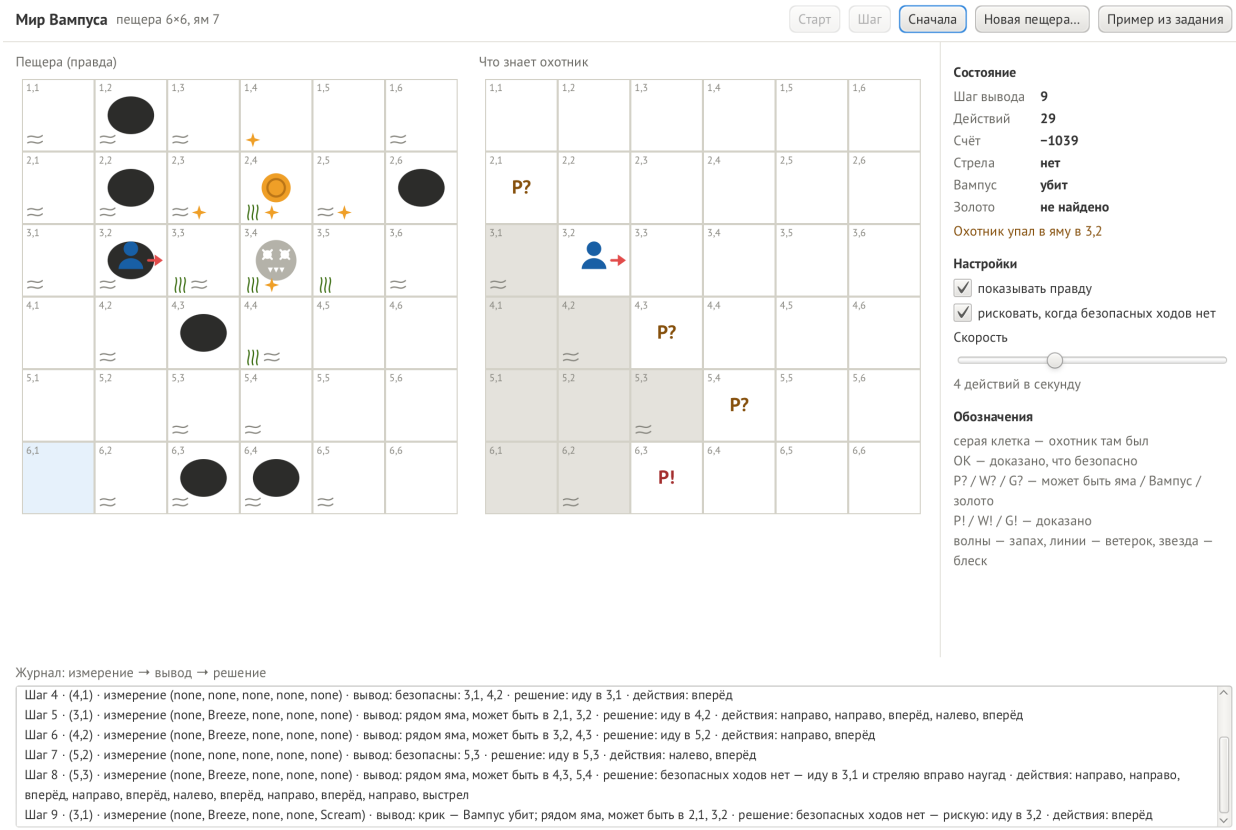


Рисунок 15 — Гибель после рискованного шага

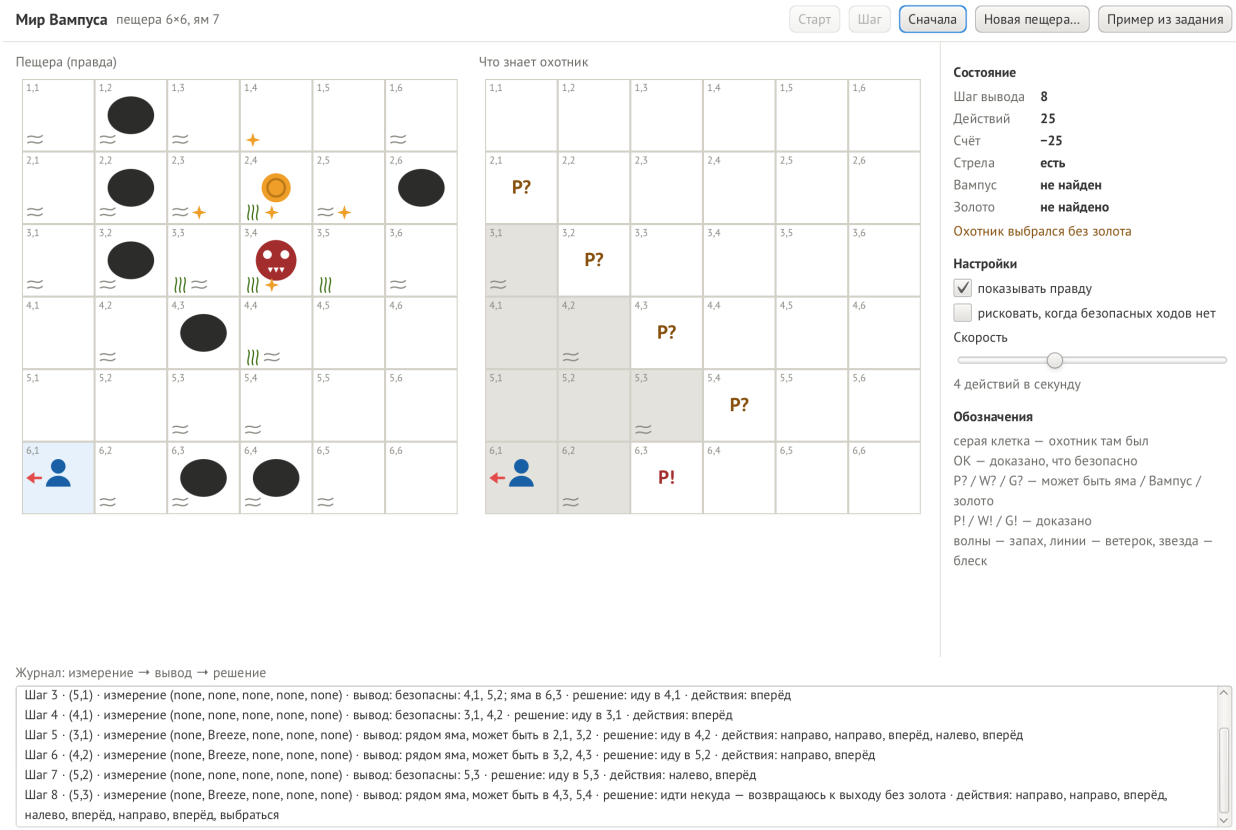


Рисунок 16 — Без риска: выход без золота

На рисунке 17 показано окно «Новая пещера» после ошибочного ввода: окно не закрылось, неверные поля подсвечены, под формой перечислены ошибки.

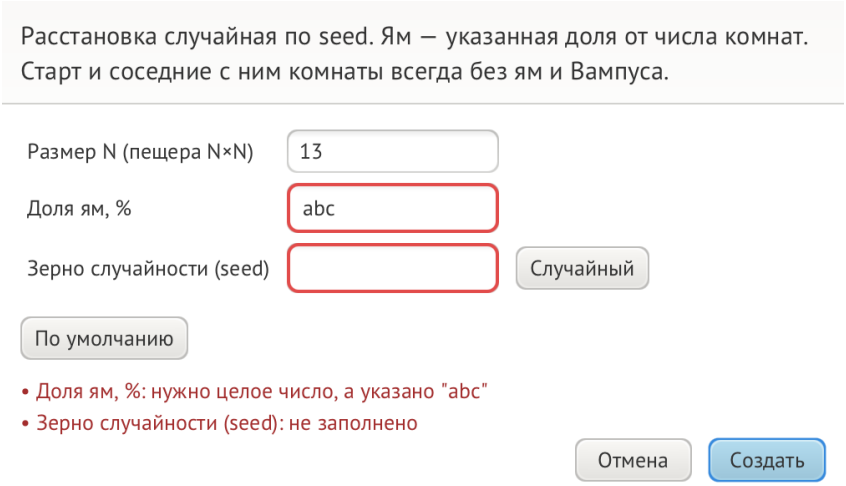


Рисунок 17 — Окно новой пещеры с ошибочным вводом

Заключение

В ходе лабораторной работы разработана программа «Мир Вампуса», в которой охотник действует как агент, основанный на знаниях. Охотник не видит карту: правила мира и показания датчиков он записывает в базу знаний в виде предложений пропозициональной логики, а безопасность комнат, положение ям, Вампуса и золота выводит проверкой логического следования с помощью алгоритма DPLL. Вывод объединяет знания, полученные на разных шагах; программа повторяет пример из задания и показывает представление агента на каждом шаге.

Пещера любого размера от 4 × 4 до 12 × 12 строится случайно, доля ям задаётся пользователем, поэтому число ям растёт пропорционально числу комнат. На 1000 случайных пещерах 6 × 6 охотник выигрывает в 45 % игр с риском и в 38 % без риска; без риска он не погибает никогда. Проигрыши объясняются не ошибками, а пределом чисто логического вывода: при большом числе ям измерений не хватает, чтобы доказать безопасность новых комнат.

Программа построена на принципах ООП и разделена на мир, логику, агента, управление и окно; мир, логика и управление не зависят от библиотеки интерфейса. Проект описан UML-диаграммами шести видов и проверен 169 автоматическими тестами.

Приложение А. Текст программы

Ниже приведён полный текст программы (без автоматических тестов). Файлы расположены в порядке: сборка и модуль, мир, логика, охотник, управление, вспомогательный класс, окно. Проект вместе с тестами, диаграммами и этим отчётом опубликован в репозитории https://github.com/azatmn/wumpus-world.

Файл pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>


    <groupId>ru.spbstu.oop</groupId>
    <artifactId>wumpus-world</artifactId>
    <version>1.0</version>
    <name>Мир Вампуса</name>


    <properties>
        <maven.compiler.release>25</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <javafx.version>25.0.4</javafx.version>
        <junit.version>6.1.3</junit.version>
    </properties>


    <dependencies>
        <dependency>
            <groupId>org.openjfx</groupId>
            <artifactId>javafx-controls</artifactId>
            <version>${javafx.version}</version>
        </dependency>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>${junit.version}</version>
            <scope>test</scope>
        </dependency>
    </dependencies>


    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.16.0</version>
            </plugin>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.6.0</version>
            </plugin>
            <plugin>
                <groupId>org.openjfx</groupId>
                <artifactId>javafx-maven-plugin</artifactId>
                <version>0.0.8</version>
                <configuration>
                    <!-- модуль/класс: программа оформлена как модуль Java -->
                    <mainClass>wumpus/wumpus.Launcher</mainClass>
```

```
                    <options>
                        <!-- разрешить JavaFX загружать системный код без предупреждения --
>
                        <option>--enable-native-access=javafx.graphics</option>
                    </options>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

Файл src/main/java/module-info.java

```
module wumpus {
    requires javafx.controls;
    exports wumpus;
    exports wumpus.model;
    exports wumpus.logic;
    exports wumpus.agent;
    exports wumpus.controller;
    exports wumpus.util;
    exports wumpus.view to javafx.graphics;
    opens wumpus.view to javafx.graphics;
}
```

Файл src/main/java/wumpus/Launcher.java

```
package wumpus;


import javafx.application.Application;
import wumpus.view.WumpusApp;


/**
 * Точка входа.
 * Отдельный класс, а не сам WumpusApp: если главный класс наследует Application,
 * запуск без модулей JavaFX падает с ошибкой «JavaFX runtime components are missing».
 */
public final class Launcher {


    private Launcher() {
    }


    public static void main(String[] args) {
        Application.launch(WumpusApp.class, args);
    }
}
```

Файл src/main/java/wumpus/model/Direction.java

```
package wumpus.model;


/** Куда смотрит охотник. Порядок — по часовой стрелке от «вверх»; строки нумеруются сверху
вниз. */
public enum Direction {
    UP(-1, 0, "вверх"),
    RIGHT(0, 1, "вправо"),
    DOWN(1, 0, "вниз"),
    LEFT(0, -1, "влево");


    private final int dRow;
    private final int dCol;
    private final String title;


    Direction(int dRow, int dCol, String title) {
        this.dRow = dRow;
        this.dCol = dCol;
        this.title = title;
    }
```

```
    public int dRow() {
        return dRow;
    }


    public int dCol() {
        return dCol;
    }


    /** Название для журнала: «вверх», «вправо»... */
    public String title() {
        return title;
    }


    /** Поворот на 90° по часовой стрелке. */
    public Direction right() {
        return values()[(ordinal() + 1) % 4];
    }


    /** Поворот на 90° против часовой стрелки. */
    public Direction left() {
        return values()[(ordinal() + 3) % 4];
    }
}
```

Файл src/main/java/wumpus/model/Position.java

```
package wumpus.model;


import java.util.ArrayList;
import java.util.List;


/** Комната пещеры в нотации задания: (строка, столбец), (1,1) — левый верхний угол. */
public record Position(int row, int col) {


    /** Соседняя комната в направлении d (без проверки границ). */
    public Position step(Direction d) {
        return new Position(row + d.dRow(), col + d.dCol());
    }


    public boolean inside(int size) {
        return row >= 1 && row <= size && col >= 1 && col <= size;
    }


    /** Соседи по сторонам в порядке вверх, вправо, вниз, влево — только внутри поля. */
    public List<Position> neighbors(int size) {
        List<Position> result = new ArrayList<>(4);
        for (Direction d : Direction.values()) {
            Position n = step(d);
            if (n.inside(size)) {
                result.add(n);
            }
        }
        return result;
    }


    public boolean isNeighbor(Position other) {
        return Math.abs(row - other.row) + Math.abs(col - other.col) == 1;
    }


    @Override
    public String toString() {
        return row + "," + col;
    }
}
```

Файл src/main/java/wumpus/model/Action.java

```
package wumpus.model;


/** Действия охотника: три привода из задания и два, без которых игру не закончить. */
public enum Action {
    FORWARD("вперёд"),
    TURN_LEFT("налево"),
    TURN_RIGHT("направо"),
    SHOOT("выстрел"),
    GRAB("взять золото"),
    CLIMB("выбраться");


    private final String title;


    Action(String title) {
        this.title = title;
    }


    public String title() {
        return title;
    }
}
```

Файл src/main/java/wumpus/model/Percept.java

```
package wumpus.model;


/** Одно измерение пяти датчиков, в порядке задания: (Stench, Breeze, Glitter, Bump,
Scream). */
public record Percept(boolean stench, boolean breeze, boolean glitter, boolean bump,
boolean scream) {


    public static final Percept NONE = new Percept(false, false, false, false, false);


    @Override
    public String toString() {
        return "(" + (stench ? "Stench" : "none")
                + ", " + (breeze ? "Breeze" : "none")
                + ", " + (glitter ? "Glitter" : "none")
                + ", " + (bump ? "Bump" : "none")
                + ", " + (scream ? "Scream" : "none") + ")";
    }
}
```

Файл src/main/java/wumpus/model/Cave.java

```
package wumpus.model;


import java.util.Objects;
import java.util.Set;


/**
 * Расстановка в пещере N×N: Вампус, золото, ямы. Не меняется; что убито и что взято,
хранит {@link World}.
 * Охотник стартует в левом нижнем углу (N,1).
 */
public final class Cave {


    public static final int MIN_SIZE = 2;
    public static final int MAX_SIZE = 12;


    private final int size;
    private final Position wumpus;
    private final Position gold;
    private final Set<Position> pits;
```

```
    public Cave(int size, Position wumpus, Position gold, Set<Position> pits) {
        if (size < MIN_SIZE || size > MAX_SIZE) {
            throw new IllegalArgumentException("Размер пещеры от " + MIN_SIZE + " до " +
MAX_SIZE + ", указано " + size);
        }
        this.size = size;
        this.wumpus = Objects.requireNonNull(wumpus);
        this.gold = Objects.requireNonNull(gold);
        this.pits = Set.copyOf(pits);
        Position start = start();
        requireInside(wumpus, "Вампус");
        requireInside(gold, "золото");
        for (Position pit : this.pits) {
            requireInside(pit, "яма");
        }
        if (wumpus.equals(start)) {
            throw new IllegalArgumentException("Вампус не может быть на старте");
        }
        if (!wumpus.isNeighbor(gold)) {
            throw new IllegalArgumentException("Золото должно лежать в соседней с Вампусом
комнате");
        }
        if (this.pits.contains(start) || this.pits.contains(wumpus) ||
this.pits.contains(gold)) {
            throw new IllegalArgumentException("Яма не может быть на старте, под Вампусом
или под золотом");
        }
    }

    /** Пещера 4×4 с картинки задания. */
    public static Cave pdfExample() {
        return new Cave(4, new Position(2, 1), new Position(2, 2),
                Set.of(new Position(1, 4), new Position(2, 3), new Position(4, 3)));
    }

    private void requireInside(Position p, String what) {
        if (!p.inside(size)) {
            throw new IllegalArgumentException(what + " вне пещеры: " + p);
        }
    }

    public int size() {
        return size;
    }

    public Position start() {
        return new Position(size, 1);
    }

    public Position wumpus() {
        return wumpus;
    }

    public Position gold() {
        return gold;
    }

    public Set<Position> pits() {
        return pits;
    }

    public boolean isPit(Position p) {
        return pits.contains(p);
    }

    /** Ветерок: яма в соседней комнате. */
    public boolean hasBreeze(Position p) {
```

```
        for (Position n : p.neighbors(size)) {
            if (pits.contains(n)) {
                return true;
            }
        }
        return false;
    }

    /** Запах: Вампус (живой или мёртвый) здесь или в соседней комнате. */
    public boolean hasStench(Position p) {
        return wumpus.equals(p) || wumpus.isNeighbor(p);
    }

    /** Золото здесь или в соседней комнате — там охотник видит блеск, пока золото не
взято. */
    public boolean nearGold(Position p) {
        return gold.equals(p) || gold.isNeighbor(p);
    }
}
```

Файл src/main/java/wumpus/model/CaveSpec.java

```
package wumpus.model;

import java.util.ArrayList;
import java.util.List;

/**
 * Заказ на случайную пещеру. Проверяется при создании; обо всех проблемах сообщает сразу.
 *
 * @param pitPercent доля комнат с ямами, в процентах: ям тем больше, чем больше пещера
 * @param seed       зерно генератора: одинаковый seed — одинаковая пещера
 */
public record CaveSpec(int size, int pitPercent, long seed) {

    public static final int MIN_SIZE = 4;
    public static final int MAX_PIT_PERCENT = 40;

    public static CaveSpec defaults() {
        return new CaveSpec(6, 20, 1L);
    }

    public CaveSpec {
        List<InvalidSettingsException.Problem> problems = new ArrayList<>();
        if (size < MIN_SIZE || size > Cave.MAX_SIZE) {
            problems.add(new InvalidSettingsException.Problem("size",
                    "Размер пещеры: допустимо от " + MIN_SIZE + " до " + Cave.MAX_SIZE + ",
указано " + size));
        }
        if (pitPercent < 0 || pitPercent > MAX_PIT_PERCENT) {
            problems.add(new InvalidSettingsException.Problem("pitPercent",
                    "Доля ям: допустимо от 0 до " + MAX_PIT_PERCENT + " %, указано " +
pitPercent));
        }
        if (!problems.isEmpty()) {
            throw new InvalidSettingsException(problems);
        }
    }

    /** Сколько ям: доля от числа комнат N², с округлением. */
    public int pitCount() {
        return (int) Math.round(size * size * pitPercent / 100.0);
    }
}
```

Файл src/main/java/wumpus/model/CaveGenerator.java

```
package wumpus.model;
```

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
import java.util.Random;
import java.util.Set;


/**
 * Случайная пещера по заказу. Старт и соседние с ним комнаты всегда чистые — иначе часть
игр
 * заканчивалась бы на первом шаге; золото кладётся рядом с Вампусом, ямы — в оставшиеся
комнаты.
 */
public final class CaveGenerator {


    private CaveGenerator() {
    }


    public static Cave generate(CaveSpec spec) {
        int size = spec.size();
        Random random = new Random(spec.seed());
        Position start = new Position(size, 1);
        Set<Position> startZone = new HashSet<>(start.neighbors(size));
        startZone.add(start);


        List<Position> free = new ArrayList<>();
        for (int row = 1; row <= size; row++) {
            for (int col = 1; col <= size; col++) {
                Position p = new Position(row, col);
                if (!startZone.contains(p)) {
                    free.add(p);
                }
            }
        }
        Position wumpus = free.get(random.nextInt(free.size()));
        List<Position> goldPlaces = wumpus.neighbors(size);
        Position gold = goldPlaces.get(random.nextInt(goldPlaces.size()));


        free.remove(wumpus);
        free.remove(gold);
        Collections.shuffle(free, random);
        Set<Position> pits = new HashSet<>(free.subList(0, spec.pitCount()));
        return new Cave(size, wumpus, gold, pits);
    }
}
```

Файл src/main/java/wumpus/model/World.java

```
package wumpus.model;


import java.util.Objects;


/**
 * Среда: выполняет действия охотника по правилам задания и возвращает измерение датчиков.
 * Охотник стартует в (N,1) лицом вправо.
 * Счёт: −1 за действие, −10 за выстрел, −1000 за смерть, +1000 за выход с золотом.
 */
public final class World {


    public static final int ACTION_COST = 1;
    public static final int ARROW_COST = 10;
    public static final int DEATH_PENALTY = 1000;
    public static final int GOLD_REWARD = 1000;


    private final Cave cave;
    private Position position;
    private Direction facing = Direction.RIGHT;
```

```
    private boolean hasArrow = true;
    private boolean wumpusAlive = true;
    private boolean hasGold;
    private boolean alive = true;
    private boolean over;
    private int score;
    private int actions;
    private Percept percept;


    public World(Cave cave) {
        this.cave = Objects.requireNonNull(cave);
        this.position = cave.start();
        this.percept = sense(false, false);
    }


    /** Выполнить действие; вернуть измерение после него. */
    public Percept execute(Action action) {
        Objects.requireNonNull(action);
        if (over) {
            throw new IllegalStateException("Игра окончена");
        }
        actions++;
        score -= ACTION_COST;
        boolean bump = false;
        boolean scream = false;
        switch (action) {
            case FORWARD -> bump = forward();
            case TURN_LEFT -> facing = facing.left();
            case TURN_RIGHT -> facing = facing.right();
            case SHOOT -> scream = shoot();
            case GRAB -> grab();
            case CLIMB -> climb();
        }
        percept = sense(bump, scream);
        return percept;
    }


    /** true — удар о стену. */
    private boolean forward() {
        Position next = position.step(facing);
        if (!next.inside(cave.size())) {
            return true;
        }
        position = next;
        if (cave.isPit(position) || (wumpusAlive && cave.wumpus().equals(position))) {
            alive = false;
            over = true;
            score -= DEATH_PENALTY;
        }
        return false;
    }


    /** Стрела летит по прямой до стены. true — Вампус убит (крик). */
    private boolean shoot() {
        if (!hasArrow) {
            return false;
        }
        hasArrow = false;
        score -= ARROW_COST;
        for (Position p = position.step(facing); p.inside(cave.size()); p = p.step(facing))
{
            if (wumpusAlive && cave.wumpus().equals(p)) {
                wumpusAlive = false;
                return true;
            }
        }
        return false;
    }


    /** Золото берётся только после убийства Вампуса — так в задании. */
    private void grab() {
        if (!hasGold && !wumpusAlive && cave.gold().equals(position)) {
```

```
            hasGold = true;
        }
    }

    private void climb() {
        if (!position.equals(cave.start())) {
            return;
        }
        over = true;
        if (hasGold) {
            score += GOLD_REWARD;
        }
    }

    private Percept sense(boolean bump, boolean scream) {
        return new Percept(cave.hasStench(position), cave.hasBreeze(position),
                !hasGold && cave.nearGold(position), bump, scream);
    }

    public Cave cave() {
        return cave;
    }

    public Position position() {
        return position;
    }

    public Direction facing() {
        return facing;
    }

    public boolean hasArrow() {
        return hasArrow;
    }

    public boolean wumpusAlive() {
        return wumpusAlive;
    }

    public boolean hasGold() {
        return hasGold;
    }

    public boolean alive() {
        return alive;
    }

    public boolean over() {
        return over;
    }

    /** Победа: выбрался из пещеры с золотом. */
    public boolean won() {
        return over && alive && hasGold;
    }

    public int score() {
        return score;
    }

    public int actions() {
        return actions;
    }

    /** Последнее измерение. */
    public Percept percept() {
```

```
        return percept;
    }
}
```

Файл src/main/java/wumpus/model/InvalidSettingsException.java

```
package wumpus.model;


import java.util.List;


/**
 * Настройки некорректны. Содержит сразу все найденные проблемы,
 * чтобы пользователь исправил их за один раз, а не по одной.
 */
public class InvalidSettingsException extends IllegalArgumentException {


    /** Одна проблема: какое поле и что с ним не так (понятным языком). */
    public record Problem(String field, String message) {
    }


    private final List<Problem> problems;


    public InvalidSettingsException(List<Problem> problems) {
        super("Некорректные настройки: " + String.join("; ",
problems.stream().map(Problem::message).toList()));
        this.problems = List.copyOf(problems);
    }


    public List<Problem> problems() {
        return problems;
    }
}
```

Файл src/main/java/wumpus/logic/Kind.java

```
package wumpus.logic;


/** О чём бывает факт: яма, Вампус или золото в комнате. Буквы — как в задании (P, W) и G
для золота. */
public enum Kind {
    PIT("P", "яма"),
    WUMPUS("W", "Вампус"),
    GOLD("G", "золото");


    private final String letter;
    private final String title;


    Kind(String letter, String title) {
        this.letter = letter;
        this.title = title;
    }


    public String letter() {
        return letter;
    }


    public String title() {
        return title;
    }
}
```

Файл src/main/java/wumpus/logic/Fact.java

```
package wumpus.logic;
```

```
import wumpus.model.Position;

import java.util.Objects;

/** Высказывание «в комнате cell есть kind», например P(3,2) — «в 3,2 яма». */
public record Fact(Kind kind, Position cell) {

    public Fact {
        Objects.requireNonNull(kind);
        Objects.requireNonNull(cell);
    }

    @Override
    public String toString() {
        return kind.letter() + "(" + cell + ")";
    }
}
```

Файл src/main/java/wumpus/logic/Literal.java

```
package wumpus.logic;

import java.util.Objects;

/** Факт или его отрицание: P(3,2) или ¬P(3,2). */
public record Literal(Fact fact, boolean positive) {

    public Literal {
        Objects.requireNonNull(fact);
    }

    public static Literal of(Fact fact) {
        return new Literal(fact, true);
    }

    public static Literal not(Fact fact) {
        return new Literal(fact, false);
    }

    public Literal negate() {
        return new Literal(fact, !positive);
    }

    @Override
    public String toString() {
        return (positive ? "" : "¬") + fact;
    }
}
```

Файл src/main/java/wumpus/logic/Clause.java

```
package wumpus.logic;

import java.util.List;
import java.util.stream.Collectors;

/** Предложение базы знаний: литералы через ИЛИ. Верно, если верен хотя бы один литерал. */
public record Clause(List<Literal> literals) {

    public Clause {
        literals = List.copyOf(literals);
        if (literals.isEmpty()) {
            throw new IllegalArgumentException("Пустое предложение всегда ложно");
        }
```

```
    }

    public Clause(Literal... literals) {
        this(List.of(literals));
    }

    @Override
    public String toString() {
        return literals.stream().map(Literal::toString).collect(Collectors.joining(" ∨ "));
    }
}
```

Файл src/main/java/wumpus/logic/Status.java

```
package wumpus.logic;

/** Что охотник может вывести о факте: точно нет, может быть, точно да. */
public enum Status {
    NO,
    MAYBE,
    YES
}
```

Файл src/main/java/wumpus/logic/Rules.java

```
package wumpus.logic;

import wumpus.model.Direction;
import wumpus.model.Position;

import java.util.ArrayList;
import java.util.List;

/**
 * Правила пещеры в виде предложений базы знаний.
 * Правило «ветерок в комнате ⇔ яма в одной из соседних» при известном измерении
превращается
 * в одно предложение (ветерок есть: P(сосед1) ∨ P(сосед2) ∨ ...) или в отрицания (ветерка
нет: ¬P у каждого соседа).
 */
public final class Rules {

    private Rules() {
    }

    /** Ветерок: яма в соседней комнате. */
    public static List<Clause> breeze(Position at, boolean felt, int size) {
        return sensor(Kind.PIT, at.neighbors(size), felt);
    }

    /** Запах: Вампус в этой или соседней комнате. */
    public static List<Clause> stench(Position at, boolean felt, int size) {
        return sensor(Kind.WUMPUS, withSelf(at, size), felt);
    }

    /** Блеск: золото в этой или соседней комнате. */
    public static List<Clause> glitter(Position at, boolean felt, int size) {
        return sensor(Kind.GOLD, withSelf(at, size), felt);
    }

    /** Охотник жив в комнате: ямы здесь нет, и живого Вампуса тоже. */
    public static List<Clause> alive(Position at, boolean wumpusAlive) {
        List<Clause> result = new ArrayList<>();
        result.add(new Clause(Literal.not(new Fact(Kind.PIT, at))));
        if (wumpusAlive) {
            result.add(new Clause(Literal.not(new Fact(Kind.WUMPUS, at))));
```

```
        }
        return result;
    }


    /** Выстрел из комнаты from: крик — Вампус на линии полёта, тишина — на линии его нет.
*/
    public static List<Clause> arrow(Position from, Direction direction, boolean scream,
int size) {
        List<Position> line = new ArrayList<>();
        for (Position p = from.step(direction); p.inside(size); p = p.step(direction)) {
            line.add(p);
        }
        if (line.isEmpty()) {
            return List.of();
        }
        return sensor(Kind.WUMPUS, line, scream);
    }


    /** Ровно один: хотя бы в одной комнате и ни в каких двух сразу. */
    public static List<Clause> exactlyOne(Kind kind, int size) {
        List<Position> cells = cells(size);
        List<Clause> result = new ArrayList<>();
        result.add(new Clause(cells.stream().map(c -> Literal.of(new Fact(kind,
c))).toList()));
        for (int i = 0; i < cells.size(); i++) {
            for (int j = i + 1; j < cells.size(); j++) {
                result.add(new Clause(Literal.not(new Fact(kind, cells.get(i))),
                        Literal.not(new Fact(kind, cells.get(j)))));
            }
        }
        return result;
    }


    /** Вампус сторожит золото: золото в комнате ⇒ Вампус в одной из соседних. */
    public static List<Clause> goldNextToWumpus(int size) {
        List<Clause> result = new ArrayList<>();
        for (Position c : cells(size)) {
            List<Literal> literals = new ArrayList<>();
            literals.add(Literal.not(new Fact(Kind.GOLD, c)));
            for (Position n : c.neighbors(size)) {
                literals.add(Literal.of(new Fact(Kind.WUMPUS, n)));
            }
            result.add(new Clause(literals));
        }
        return result;
    }


    /** Ни Вампус, ни золото не лежат в яме. */
    public static List<Clause> notInPit(int size) {
        List<Clause> result = new ArrayList<>();
        for (Position c : cells(size)) {
            Literal noPit = Literal.not(new Fact(Kind.PIT, c));
            result.add(new Clause(Literal.not(new Fact(Kind.WUMPUS, c)), noPit));
            result.add(new Clause(Literal.not(new Fact(Kind.GOLD, c)), noPit));
        }
        return result;
    }


    private static List<Clause> sensor(Kind kind, List<Position> cells, boolean felt) {
        if (felt) {
            return List.of(new Clause(cells.stream().map(c -> Literal.of(new Fact(kind,
c))).toList()));
        }
        return cells.stream().map(c -> new Clause(Literal.not(new Fact(kind,
c)))).toList();
    }


    private static List<Position> withSelf(Position at, int size) {
        List<Position> result = new ArrayList<>();
        result.add(at);
        result.addAll(at.neighbors(size));
```

```
        return result;
    }

    /** Все комнаты построчно. */
    static List<Position> cells(int size) {
        List<Position> result = new ArrayList<>(size * size);
        for (int row = 1; row <= size; row++) {
            for (int col = 1; col <= size; col++) {
                result.add(new Position(row, col));
            }
        }
        return result;
    }
}
```

Файл src/main/java/wumpus/logic/KnowledgeBase.java

```
package wumpus.logic;

import wumpus.model.Cave;
import wumpus.model.Position;

import java.util.ArrayList;
import java.util.Collection;
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * База знаний охотника: набор предложений. TELL — {@link #tell}, ASK — {@link #entails} и
{@link #status}.
 * <p>
 * Факт выводим (KB ⊨ α), если «база знаний и не α» невыполнимо: ни одна расстановка,
согласная с базой,
 * не делает α ложным. Выполнимость проверяет {@link Dpll}.
 * С самого начала в базе — правила мира: Вампус и золото ровно одни, золото рядом с
Вампусом, оба не в яме.
 */
public final class KnowledgeBase {

    private final int size;
    private final List<Clause> clauses = new ArrayList<>();
    private final List<int[]> encoded = new ArrayList<>();
    /** Точные выводы не меняются: база только растёт. */
    private final Map<Fact, Status> settled = new HashMap<>();
    private int solverCalls;

    public KnowledgeBase(int size) {
        if (size < Cave.MIN_SIZE || size > Cave.MAX_SIZE) {
            throw new IllegalArgumentException("Размер пещеры от " + Cave.MIN_SIZE + " до "
+ Cave.MAX_SIZE + ", указано " + size);
        }
        this.size = size;
        tell(Rules.exactlyOne(Kind.WUMPUS, size));
        tell(Rules.exactlyOne(Kind.GOLD, size));
        tell(Rules.goldNextToWumpus(size));
        tell(Rules.notInPit(size));
    }

    public int size() {
        return size;
    }

    public void tell(Clause clause) {
        int[] c = new int[clause.literals().size()];
        for (int i = 0; i < c.length; i++) {
            c[i] = encode(clause.literals().get(i));
        }
```

```
        clauses.add(clause);
        encoded.add(c);
    }

    public void tell(Collection<Clause> list) {
        for (Clause c : list) {
            tell(c);
        }
    }

    public List<Clause> clauses() {
        return List.copyOf(clauses);
    }

    /** Следует ли литерал из базы: база вместе с его отрицанием невыполнима. */
    public boolean entails(Literal literal) {
        return solveWith(-encode(literal)) == null;
    }

    public Status status(Fact fact) {
        return statuses(List.of(fact)).get(fact);
    }

    /**
     * Статус каждого факта. Найденная расстановка показывает возможные значения сразу
многих фактов,
     * поэтому DPLL запускается только для тех, чьё значение ещё ни разу не встретилось.
     */
    public Map<Fact, Status> statuses(Collection<Fact> facts) {
        Map<Fact, Status> result = new LinkedHashMap<>();
        List<boolean[]> models = new ArrayList<>();
        for (Fact fact : facts) {
            int v = variable(fact);
            Status known = settled.get(fact);
            if (known != null) {
                result.put(fact, known);
                continue;
            }
            boolean canBeTrue = possible(v, models);
            boolean canBeFalse = possible(-v, models);
            if (!canBeTrue && !canBeFalse) {
                throw new IllegalStateException("База знаний противоречива");
            }
            Status s = canBeTrue && canBeFalse ? Status.MAYBE : canBeTrue ? Status.YES :
Status.NO;
            if (s != Status.MAYBE) {
                settled.put(fact, s);
            }
            result.put(fact, s);
        }
        return result;
    }

    /** Есть ли расстановка с литералом lit: сначала среди найденных, иначе спросить DPLL.
*/
    private boolean possible(int lit, List<boolean[]> models) {
        for (boolean[] m : models) {
            if (m[Math.abs(lit)] == lit > 0) {
                return true;
            }
        }
        boolean[] m = solveWith(lit);
        if (m == null) {
            return false;
        }
        models.add(m);
        return true;
    }

    private boolean[] solveWith(int lit) {
```

```
        solverCalls++;
        List<int[]> formula = new ArrayList<>(encoded.size() + 1);
        formula.addAll(encoded);
        formula.add(new int[]{lit});
        return Dpll.solve(3 * size * size, formula);
    }

    /** Сколько раз запускался DPLL — для проверки, что найденные расстановки
переиспользуются. */
    int solverCalls() {
        return solverCalls;
    }

    private int encode(Literal literal) {
        int v = variable(literal.fact());
        return literal.positive() ? v : -v;
    }

    /** Номер переменной: сначала все ямы, потом все Вампусы, потом всё золото. */
    private int variable(Fact fact) {
        Position c = fact.cell();
        if (!c.inside(size)) {
            throw new IllegalArgumentException("Комната вне пещеры: " + c);
        }
        return fact.kind().ordinal() * size * size + (c.row() - 1) * size + c.col();
    }
}
```

Файл src/main/java/wumpus/logic/Dpll.java

```
package wumpus.logic;

import java.util.ArrayList;
import java.util.List;

/**
 * Проверка выполнимости формулы в КНФ: DPLL с обучением на конфликтах (CDCL).
 * <p>
 * Переменные — числа 1..n, литерал x — «переменная x истинна», −x — «ложна».
 * Предложение — массив литералов, соединённых ИЛИ; формула — список предложений,
соединённых И.
 * <p>
 * Как обычный DPLL: выбирает значение переменной, выводит следствия из единичных
предложений,
 * при противоречии откатывается. Отличие: из противоречия выводится новое предложение —
его причина,
 * и откат идёт сразу к решению, которое к ней привело. Без этого перебор раз за разом
 * переставляет не относящиеся к делу переменные и на пещерах побольше не заканчивается.
 * Каждое предложение следит за двумя своими литералами и проверяется, только когда один из
них стал ложным.
 */
public final class Dpll {

    private static final int NO_REASON = -1;

    private final List<int[]> clauses = new ArrayList<>();
    private final List<List<Integer>> watchers;
    private final int[] value;
    private final int[] level;
    private final int[] reason;
    private final int[] trail;
    private int trailSize;
    private int propagated;
    /** Где на следе начинается каждый уровень решений. */
    private final List<Integer> levelStart = new ArrayList<>();
    private final boolean[] seen;

    private Dpll(int varCount, List<int[]> formula) {
        value = new int[varCount + 1];
```

```
        level = new int[varCount + 1];
        reason = new int[varCount + 1];
        seen = new boolean[varCount + 1];
        trail = new int[varCount];
        watchers = new ArrayList<>(2 * varCount + 2);
        for (int i = 0; i < 2 * varCount + 2; i++) {
            watchers.add(new ArrayList<>());
        }
        for (int[] c : formula) {
            for (int lit : c) {
                if (lit == 0 || Math.abs(lit) > varCount) {
                    throw new IllegalArgumentException("Литерал " + lit + " вне переменных
1.." + varCount);
                }
            }
        }
        for (int[] c : formula) {
            clauses.add(c.clone());
        }
    }


    /**
     * Найти набор значений, при котором верны все предложения.
     *
     * @return model[x] — значение переменной x (model[0] не используется) или null, если
формула невыполнима
     */
    public static boolean[] solve(int varCount, List<int[]> formula) {
        return new Dpll(varCount, formula).search();
    }


    private static int index(int lit) {
        return lit > 0 ? 2 * lit : -2 * lit + 1;
    }


    /** 1 — литерал истинен, -1 — ложен, 0 — переменная ещё не задана. */
    private int valueOf(int lit) {
        int v = value[Math.abs(lit)];
        return lit > 0 ? v : -v;
    }


    private int currentLevel() {
        return levelStart.size();
    }


    private void assign(int lit, int why) {
        int v = Math.abs(lit);
        value[v] = lit > 0 ? 1 : -1;
        level[v] = currentLevel();
        reason[v] = why;
        trail[trailSize++] = lit;
    }


    /** Откатить все решения выше уровня target. */
    private void backjump(int target) {
        int size = levelStart.get(target);
        while (trailSize > size) {
            value[Math.abs(trail[--trailSize])] = 0;
        }
        propagated = size;
        while (levelStart.size() > target) {
            levelStart.removeLast();
        }
    }


    private void watch(int ci) {
        int[] c = clauses.get(ci);
        watchers.get(index(c[0])).add(ci);
        watchers.get(index(c[1])).add(ci);
    }
```

```
    /** Подготовить слежение и единичные предложения. false — противоречие уже на старте.
*/
    private boolean init() {
        for (int i = 0; i < clauses.size(); i++) {
            int[] c = clauses.get(i);
            if (c.length == 0) {
                return false;
            }
            if (c.length == 1) {
                int v = valueOf(c[0]);
                if (v < 0) {
                    return false;
                }
                if (v == 0) {
                    assign(c[0], NO_REASON);
                }
                continue;
            }
            watch(i);
        }
        return true;
    }


    /** Распространение единичных предложений. Возвращает номер нарушенного предложения или
-1. */
    private int propagate() {
        while (propagated < trailSize) {
            int falseLit = -trail[propagated++];
            List<Integer> list = watchers.get(index(falseLit));
            for (int w = 0; w < list.size(); ) {
                int ci = list.get(w);
                int[] c = clauses.get(ci);
                if (c[0] == falseLit) {
                    c[0] = c[1];
                    c[1] = falseLit;
                }
                if (valueOf(c[0]) > 0) {
                    w++;
                    continue;
                }
                boolean moved = false;
                for (int k = 2; k < c.length; k++) {
                    if (valueOf(c[k]) >= 0) {
                        c[1] = c[k];
                        c[k] = falseLit;
                        watchers.get(index(c[1])).add(ci);
                        list.set(w, list.getLast());
                        list.removeLast();
                        moved = true;
                        break;
                    }
                }
                if (moved) {
                    continue;
                }
                int other = valueOf(c[0]);
                if (other < 0) {
                    return ci;
                }
                if (other == 0) {
                    assign(c[0], ci);
                }
                w++;
            }
        }
        return -1;
    }


    /**
     * Причина противоречия — новое предложение (схема «первой точки сочленения»):
     * разворачиваем следствия текущего уровня назад, пока на нём не останется один
литерал.
```

```
 * Первым в результате стоит отрицание этого литерала.
 */
private int[] analyze(int conflict) {
    List<Integer> learnt = new ArrayList<>();
    learnt.add(0);
    int pending = 0;
    int lit = 0;
    int ci = conflict;
    int t = trailSize - 1;
    do {
        for (int q : clauses.get(ci)) {
            int v = Math.abs(q);
            if (q == lit || seen[v] || level[v] == 0) {
                continue;
            }
            seen[v] = true;
            if (level[v] == currentLevel()) {
                pending++;
            } else {
                learnt.add(q);
            }
        }
        while (!seen[Math.abs(trail[t])]) {
            t--;
        }
        lit = trail[t--];
        seen[Math.abs(lit)] = false;
        ci = reason[Math.abs(lit)];
        pending--;
    } while (pending > 0);
    learnt.set(0, -lit);
    int[] result = new int[learnt.size()];
    for (int i = 0; i < result.length; i++) {
        result[i] = learnt.get(i);
        seen[Math.abs(result[i])] = false;
    }
    return result;
}


private boolean[] search() {
    if (!init()) {
        return null;
    }
    int next = 1;
    while (true) {
        int conflict = propagate();
        if (conflict >= 0) {
            if (currentLevel() == 0) {
                return null;
            }
            int[] learnt = analyze(conflict);
            // откат на самый высокий уровень среди остальных литералов причины
            int target = 0;
            int at = 1;
            for (int i = 1; i < learnt.length; i++) {
                if (level[Math.abs(learnt[i])] > target) {
                    target = level[Math.abs(learnt[i])];
                    at = i;
                }
            }
            backjump(target);
            if (learnt.length == 1) {
                assign(learnt[0], NO_REASON);
            } else {
                int swap = learnt[1];
                learnt[1] = learnt[at];
                learnt[at] = swap;
                clauses.add(learnt);
                watch(clauses.size() - 1);
                assign(learnt[0], clauses.size() - 1);
            }
            next = 1;
            continue;
        }
```

```
            while (next < value.length && value[next] != 0) {
                next++;
            }
            if (next == value.length) {
                boolean[] model = new boolean[value.length];
                for (int v = 1; v < value.length; v++) {
                    model[v] = value[v] > 0;
                }
                return model;
            }
            // сначала «ложь»: в пещере почти все факты ложны (ям мало, Вампус один)
            levelStart.add(trailSize);
            assign(-next, NO_REASON);
        }
    }
}
```

Файл src/main/java/wumpus/agent/CellKnowledge.java

```
package wumpus.agent;


import wumpus.logic.Status;
import wumpus.model.Percept;
import wumpus.model.Position;


/**
 * Что охотник знает о комнате — то, что рисуется на его карте.
 *
 * @param percept измерение в комнате или null, если охотник там не был
 * @param safe    комната безопасна: ямы точно нет и живого Вампуса точно нет (пометка OK)
 */
public record CellKnowledge(Position cell, Percept percept, Status pit, Status wumpus,
Status gold, boolean safe) {


    public boolean visited() {
        return percept != null;
    }
}
```

Файл src/main/java/wumpus/agent/Turn.java

```
package wumpus.agent;


import wumpus.model.Action;
import wumpus.model.Percept;
import wumpus.model.Position;


import java.util.List;
import java.util.stream.Collectors;


/**
 * Один шаг вывода для журнала, как в задании: измерение → вывод → решение.
 *
 * @param actions действия, которыми решение выполняется: повороты, шаги, выстрел
 * @param risky   решение принято без доказанно безопасного хода
 */
public record Turn(int number, Position at, Percept percept, List<String> conclusions,
String decision,
                   List<Action> actions, boolean risky) {


    public Turn {
        conclusions = List.copyOf(conclusions);
        actions = List.copyOf(actions);
    }


    @Override
    public String toString() {
        return "Шаг " + number + " · (" + at + ") · измерение " + percept
```

```
                + " · вывод: " + (conclusions.isEmpty() ? "нового нет" : String.join("; ",
conclusions))
                + " · решение: " + decision
                + " · действия: " +
actions.stream().map(Action::title).collect(Collectors.joining(", "));
    }
}
```

Файл src/main/java/wumpus/agent/Planner.java

```
package wumpus.agent;


import wumpus.model.Action;
import wumpus.model.Direction;
import wumpus.model.Position;


import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Deque;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;


/** Маршруты охотника: поиск в ширину по разрешённым комнатам и перевод маршрута в повороты
и шаги. */
public final class Planner {


    private Planner() {
    }


    /**
     * Кратчайший путь от from до to; промежуточные комнаты — только из allowed, сама to
может быть любой.
     *
     * @return комнаты пути без from (пустой список, если from = to) или null, если пути
нет
     */
    public static List<Position> path(Position from, Position to, Set<Position> allowed,
int size) {
        if (from.equals(to)) {
            return List.of();
        }
        Map<Position, Position> cameFrom = new HashMap<>();
        Deque<Position> queue = new ArrayDeque<>();
        cameFrom.put(from, from);
        queue.add(from);
        while (!queue.isEmpty()) {
            Position p = queue.poll();
            for (Position n : p.neighbors(size)) {
                if (cameFrom.containsKey(n)) {
                    continue;
                }
                if (n.equals(to)) {
                    cameFrom.put(n, p);
                    return unwind(cameFrom, from, to);
                }
                if (allowed.contains(n)) {
                    cameFrom.put(n, p);
                    queue.add(n);
                }
            }
        }
        return null;
    }


    private static List<Position> unwind(Map<Position, Position> cameFrom, Position from,
Position to) {
        List<Position> result = new ArrayList<>();
```

```
        for (Position p = to; !p.equals(from); p = cameFrom.get(p)) {
            result.add(p);
        }
        Collections.reverse(result);
        return result;
    }


    /**
     * Самая дешёвая по числу действий цель из targets: путь к ней идёт только по allowed.
     *
     * @return путь к выбранной цели или null, если ни одна не достижима
     */
    public static List<Position> cheapest(Position from, Direction facing, List<Position>
targets,
                                          Set<Position> allowed, int size) {
        List<Position> best = null;
        int bestCost = Integer.MAX_VALUE;
        for (Position t : targets) {
            List<Position> p = path(from, t, allowed, size);
            if (p == null) {
                continue;
            }
            int cost = walk(from, facing, p).size();
            if (cost < bestCost) {
                best = p;
                bestCost = cost;
            }
        }
        return best;
    }


    /** Повороты, чтобы смотреть из from в to: меньшим числом поворотов. */
    public static List<Action> face(Direction from, Direction to) {
        int turns = (to.ordinal() - from.ordinal() + 4) % 4;
        return switch (turns) {
            case 0 -> List.of();
            case 1 -> List.of(Action.TURN_RIGHT);
            case 2 -> List.of(Action.TURN_RIGHT, Action.TURN_RIGHT);
            default -> List.of(Action.TURN_LEFT);
        };
    }


    /** Действия, чтобы пройти путь: перед каждым шагом — поворот в нужную сторону. */
    public static List<Action> walk(Position from, Direction facing, List<Position> path) {
        List<Action> result = new ArrayList<>();
        Position at = from;
        Direction looking = facing;
        for (Position next : path) {
            Direction d = direction(at, next);
            result.addAll(face(looking, d));
            result.add(Action.FORWARD);
            looking = d;
            at = next;
        }
        return result;
    }


    /** Куда охотник смотрит, пройдя путь. */
    public static Direction facingAfter(Position from, Direction facing, List<Position>
path) {
        if (path.isEmpty()) {
            return facing;
        }
        Position before = path.size() > 1 ? path.get(path.size() - 2) : from;
        return direction(before, path.getLast());
    }


    /** Направление от комнаты к соседней. */
    public static Direction direction(Position from, Position to) {
        for (Direction d : Direction.values()) {
            if (from.step(d).equals(to)) {
                return d;
```

```
            }
        }
        throw new IllegalArgumentException(to + " не соседняя с " + from);
    }
}
```

Файл src/main/java/wumpus/agent/Hunter.java

```
package wumpus.agent;


import wumpus.logic.Fact;
import wumpus.logic.Kind;
import wumpus.logic.KnowledgeBase;
import wumpus.logic.Rules;
import wumpus.logic.Status;
import wumpus.model.Action;
import wumpus.model.Direction;
import wumpus.model.Percept;
import wumpus.model.Position;


import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Deque;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.stream.Collectors;


/**
 * Охотник — агент, основанный на знаниях. Карты не видит; знает правила, свои координаты и
куда смотрит.
 * Каждое новое измерение записывает в базу знаний (TELL), спрашивает её о комнатах (ASK) и
решает, что делать.
 * <p>
 * Порядок целей: с золотом — к выходу; Вампус убит и золото найдено — за золотом; Вампус
найден — застрелить;
 * иначе — в ближайшую безопасную комнату. Безопасных нет — выстрел наугад, затем риск,
затем выход.
 */
public final class Hunter {


    private final int size;
    private final boolean takeRisks;
    private final Position start;
    private final KnowledgeBase kb;
    private Position position;
    private Direction facing = Direction.RIGHT;
    private boolean hasArrow = true;
    private boolean wumpusDead;
    private boolean hasGold;
    private boolean finished;
    private Action last;
    private Percept lastPercept;
    private final Map<Position, Percept> visited = new LinkedHashMap<>();
    private final Deque<Action> plan = new ArrayDeque<>();
    private final List<Turn> journal = new ArrayList<>();
    private final List<String> pending = new ArrayList<>();
    private Map<Position, CellKnowledge> knowledge = new LinkedHashMap<>();
    /** Куда ведёт текущий план — для текста решения. */
    private Position walkTarget;
    private Position shotSpot;
    private Direction shotDir;


    public Hunter(int size) {
        this(size, true);
    }
```

```
    /** @param takeRisks идти в комнату с недоказанной опасностью, когда безопасных не
осталось; иначе — к выходу */
    public Hunter(int size, boolean takeRisks) {
        this.size = size;
        this.takeRisks = takeRisks;
        this.kb = new KnowledgeBase(size);
        this.start = new Position(size, 1);
        this.position = start;
        refresh();
    }


    /** Получить измерение после прошлого действия и выбрать следующее. */
    public Action next(Percept percept) {
        if (finished) {
            throw new IllegalStateException("Охотник уже выбрался из пещеры");
        }
        perceive(percept);
        think();
        Action action = plan.poll();
        switch (action) {
            case TURN_LEFT -> facing = facing.left();
            case TURN_RIGHT -> facing = facing.right();
            case SHOOT -> hasArrow = false;
            case CLIMB -> finished = true;
            default -> {
            }
        }
        last = action;
        return action;
    }


    /**
     * Последствия прошлого действия и новое измерение — в базу знаний. Вызывать сразу
после действия
     * (тогда охотник сразу «слышит» крик и знает, что взял золото); повторный вызов ничего
не делает.
     */
    public void perceive(Percept percept) {
        if (last == null && !visited.isEmpty()) {
            return;
        }
        lastPercept = percept;
        if (last == Action.FORWARD && !percept.bump()) {
            position = position.step(facing);
        }
        if (percept.bump()) {
            plan.clear();
        }
        if (last == Action.SHOOT) {
            kb.tell(Rules.arrow(position, facing, percept.scream(), size));
            if (percept.scream()) {
                wumpusDead = true;
                pending.add("крик — Вампус убит");
            } else {
                pending.add("крика нет — Вампуса " + facing.title() + " нет");
            }
        }
        if (last == Action.GRAB && !percept.glitter()) {
            hasGold = true;
            pending.add("золото у охотника");
        }
        if (!visited.containsKey(position)) {
            visited.put(position, percept);
            kb.tell(Rules.alive(position, !wumpusDead));
            kb.tell(Rules.stench(position, percept.stench(), size));
            kb.tell(Rules.breeze(position, percept.breeze(), size));
            if (!hasGold) {
                kb.tell(Rules.glitter(position, percept.glitter(), size));
            }
        }
        last = null;
    }
```

```
    /** Подумать: если прошлое решение выполнено — сделать вывод, выбрать цель и записать
шаг в журнал. */
    public void think() {
        if (finished || !plan.isEmpty()) {
            return;
        }
        decide(lastPercept);
    }


    private void decide(Percept percept) {
        Map<Position, CellKnowledge> before = knowledge;
        refresh();
        List<String> conclusions = new ArrayList<>(pending);
        pending.clear();
        conclusions.addAll(conclusions(before, percept));
        String decision;
        boolean risky = false;
        Position gold = located(Kind.GOLD);
        Position wumpus = located(Kind.WUMPUS);
        if (hasGold) {
            decision = goTo(start, "золото у меня — возвращаюсь к выходу");
            plan.add(Action.CLIMB);
        } else if (wumpusDead && gold != null && pathTo(gold) != null) {
            decision = position.equals(gold) ? "беру золото" : goTo(gold, "иду за золотом в
" + gold);
            plan.add(Action.GRAB);
        } else if (!wumpusDead && !hasArrow) {
            decision = goTo(start, "стрелы нет, а Вампус жив — золото не взять, возвращаюсь
к выходу");
            plan.add(Action.CLIMB);
        } else if (!wumpusDead && wumpus != null && planShot(List.of(wumpus))) {
            decision = "Вампус в " + wumpus + " — " + shotText();
        } else if (planExplore()) {
            decision = "иду в " + target();
        } else if (takeRisks && !wumpusDead && planShot(maybe(Kind.WUMPUS))) {
            decision = "безопасных ходов нет — " + shotText() + " наугад";
            risky = true;
        } else if (takeRisks && planRisk()) {
            decision = "безопасных ходов нет — рискую: иду в " + target();
            risky = true;
        } else {
            decision = goTo(start, "идти некуда — возвращаюсь к выходу без золота");
            plan.add(Action.CLIMB);
        }
        journal.add(new Turn(journal.size() + 1, position, percept, conclusions, decision,
List.copyOf(plan), risky));
    }


    /** Пересчитать знания о всех комнатах: один запрос к базе на все факты. */
    private void refresh() {
        List<Fact> facts = new ArrayList<>();
        for (Position c : cells()) {
            for (Kind kind : Kind.values()) {
                facts.add(new Fact(kind, c));
            }
        }
        Map<Fact, Status> s = kb.statuses(facts);
        Map<Position, CellKnowledge> result = new LinkedHashMap<>();
        for (Position c : cells()) {
            Status pit = s.get(new Fact(Kind.PIT, c));
            Status w = s.get(new Fact(Kind.WUMPUS, c));
            Status g = s.get(new Fact(Kind.GOLD, c));
            boolean safe = pit == Status.NO && (w == Status.NO || wumpusDead);
            result.put(c, new CellKnowledge(c, visited.get(c), pit, w, g, safe));
        }
        knowledge = result;
    }


    /** Что нового выведено с прошлого шага — словами, как в задании. */
    private List<String> conclusions(Map<Position, CellKnowledge> before, Percept percept)
{
        List<String> result = new ArrayList<>();
        List<Position> newlySafe = new ArrayList<>();
```

```
        for (CellKnowledge k : knowledge.values()) {
            CellKnowledge was = before.get(k.cell());
            if (k.safe() && !k.visited() && !was.safe()) {
                newlySafe.add(k.cell());
            }
        }
        if (!newlySafe.isEmpty()) {
            result.add("безопасны: " + list(newlySafe));
        }
        for (Kind kind : Kind.values()) {
            for (CellKnowledge k : knowledge.values()) {
                if (status(k, kind) == Status.YES && status(before.get(k.cell()), kind) !=
Status.YES) {
                    result.add(kind.title() + " в " + k.cell());
                }
            }
        }
        if (percept.breeze()) {
            sensed(result, "рядом яма, может быть в ", Kind.PIT);
        }
        if (percept.stench()) {
            sensed(result, "рядом Вампус, может быть в ", Kind.WUMPUS);
        }
        if (percept.glitter() && !hasGold) {
            sensed(result, "рядом золото, может быть в ", Kind.GOLD);
        }
        return result;
    }


    private void sensed(List<String> result, String text, Kind kind) {
        List<Position> around = new ArrayList<>();
        around.add(position);
        around.addAll(position.neighbors(size));
        List<Position> maybe = around.stream().filter(c -> status(knowledge.get(c), kind)
== Status.MAYBE).toList();
        if (!maybe.isEmpty()) {
            result.add(text + list(maybe));
        }
    }


    private static Status status(CellKnowledge k, Kind kind) {
        return switch (kind) {
            case PIT -> k.pit();
            case WUMPUS -> k.wumpus();
            case GOLD -> k.gold();
        };
    }


    /** Комната, где факт точно верен, или null. */
    private Position located(Kind kind) {
        for (CellKnowledge k : knowledge.values()) {
            if (status(k, kind) == Status.YES) {
                return k.cell();
            }
        }
        return null;
    }


    private List<Position> maybe(Kind kind) {
        return knowledge.values().stream().filter(k -> status(k, kind) ==
Status.MAYBE).map(CellKnowledge::cell).toList();
    }


    private List<Position> pathTo(Position to) {
        return Planner.path(position, to, visited.keySet(), size);
    }


    /** Дойти по посещённым комнатам; текст решения — если идти не надо, то «здесь». */
    private String goTo(Position to, String text) {
        plan.addAll(Planner.walk(position, facing, pathTo(to)));
        return text;
```

```
    }

    /** Ближайшая безопасная непосещённая комната. */
    private boolean planExplore() {
        List<Position> targets = knowledge.values().stream()
                .filter(k -> k.safe() && !k.visited()).map(CellKnowledge::cell).toList();
        return planWalk(targets);
    }

    /**
     * Ближайшая непосещённая комната, где яма не доказана. Живой найденный Вампус сюда не
попадает:
     * его запах чувствуется в соседней посещённой комнате на одной линии с ним, и раньше
идёт выстрел.
     */
    private boolean planRisk() {
        List<Position> targets = knowledge.values().stream()
                .filter(k -> !k.visited() && k.pit() !=
Status.YES).map(CellKnowledge::cell).toList();
        return planWalk(targets);
    }

    private boolean planWalk(List<Position> targets) {
        List<Position> path = Planner.cheapest(position, facing, targets, visited.keySet(),
size);
        if (path == null || path.isEmpty()) {
            return false;
        }
        walkTarget = path.getLast();
        plan.addAll(Planner.walk(position, facing, path));
        return true;
    }

    private String target() {
        return walkTarget.toString();
    }

    /**
     * Выстрел по одной из комнат targets: из ближайшей посещённой комнаты на одной линии с
ней.
     * Из нескольких направлений — то, где на линии больше всего целей.
     */
    private boolean planShot(List<Position> targets) {
        if (targets.isEmpty()) {
            return false;
        }
        Set<Position> goals = new LinkedHashSet<>(targets);
        List<Position> spots = new ArrayList<>(visited.keySet());
        Position bestSpot = null;
        Direction bestDir = null;
        int bestHits = 0;
        int bestCost = Integer.MAX_VALUE;
        for (Position spot : spots) {
            List<Position> path = pathTo(spot);
            if (path == null) {
                continue;
            }
            for (Direction d : Direction.values()) {
                int hits = 0;
                for (Position p = spot.step(d); p.inside(size); p = p.step(d)) {
                    if (goals.contains(p)) {
                        hits++;
                    }
                }
                if (hits == 0) {
                    continue;
                }
                Direction arrive = Planner.facingAfter(position, facing, path);
                int cost = Planner.walk(position, facing, path).size() +
Planner.face(arrive, d).size();
                if (hits > bestHits || (hits == bestHits && cost < bestCost)) {
                    bestSpot = spot;
```

```
                    bestDir = d;
                    bestHits = hits;
                    bestCost = cost;
                }
            }
        }
        if (bestSpot == null) {
            return false;
        }
        List<Position> path = pathTo(bestSpot);
        plan.addAll(Planner.walk(position, facing, path));
        plan.addAll(Planner.face(Planner.facingAfter(position, facing, path), bestDir));
        plan.add(Action.SHOOT);
        shotSpot = bestSpot;
        shotDir = bestDir;
        return true;
    }

    /** «стреляю вверх» или, если стрелять из другой комнаты, «иду в 12,6 и стреляю вверх».
*/
    private String shotText() {
        String shot = "стреляю " + shotDir.title();
        return shotSpot.equals(position) ? shot : "иду в " + shotSpot + " и " + shot;
    }

    private List<Position> cells() {
        List<Position> result = new ArrayList<>(size * size);
        for (int row = 1; row <= size; row++) {
            for (int col = 1; col <= size; col++) {
                result.add(new Position(row, col));
            }
        }
        return result;
    }

    private static String list(List<Position> cells) {
        return cells.stream().map(Position::toString).collect(Collectors.joining(", "));
    }

    public Position position() {
        return position;
    }

    public Direction facing() {
        return facing;
    }

    public boolean hasArrow() {
        return hasArrow;
    }

    public boolean wumpusDead() {
        return wumpusDead;
    }

    public boolean hasGold() {
        return hasGold;
    }

    /** Сколько действий ещё осталось от текущего решения. */
    public int planned() {
        return plan.size();
    }

    public boolean finished() {
        return finished;
    }
```

```
    /** Где охотник точно знает Вампуса / золото; null — не знает. */
    public Position knownWumpus() {
        return located(Kind.WUMPUS);
    }


    public Position knownGold() {
        return located(Kind.GOLD);
    }


    /** Измерение в комнате, где охотник уже был (в том числе после последнего вывода), или
null. */
    public Percept perceptAt(Position cell) {
        return visited.get(cell);
    }


    public Map<Position, CellKnowledge> knowledge() {
        return Collections.unmodifiableMap(knowledge);
    }


    public List<Turn> journal() {
        return List.copyOf(journal);
    }


    public int clauseCount() {
        return kb.clauses().size();
    }
}
```

Файл src/main/java/wumpus/controller/GameController.java

```
package wumpus.controller;


import wumpus.agent.Hunter;
import wumpus.model.Cave;
import wumpus.model.CaveGenerator;
import wumpus.model.CaveSpec;
import wumpus.model.World;


import java.util.Objects;


/**
 * Управление игрой для окна: старт, пауза, шаг вывода, сначала, новая пещера, пример из
задания.
 * Не знает ничего про JavaFX — окно только вызывает методы и рисует состояние,
 * поэтому всю логику кнопок можно проверить обычными тестами.
 * <p>
 * Время приходит снаружи через {@link #tick(long)} (окно вызывает его на каждом кадре),
 * а контроллер сам решает, сколько действий сделать, исходя из скорости.
 */
public class GameController {


    public static final int MIN_SPEED = 1;
    public static final int MAX_SPEED = 50;
    public static final int DEFAULT_SPEED = 4;


    private static final long NANOS_PER_SECOND = 1_000_000_000L;


    private CaveSpec spec = CaveSpec.defaults();
    private Cave cave;
    private World world;
    private Hunter hunter;
    private boolean takeRisks = true;
    private boolean running;
    private int speed = DEFAULT_SPEED;
    /** Время прошлого кадра; пока его нет (сразу после старта), первый кадр только
запоминает время. */
    private boolean hasLastTick;
```

```
    private long lastTick;
    /** Накопленное время × скорость: целая часть от деления на секунду — сколько действий
пора сделать. */
    private long actionDebt;


    /** Начинает с пещеры из задания. */
    public GameController() {
        this(Cave.pdfExample());
    }


    public GameController(Cave cave) {
        this.cave = Objects.requireNonNull(cave, "cave");
        restart();
    }


    /** Кнопка «Сначала»: та же пещера, охотник ничего не знает. */
    public void restart() {
        pause();
        world = new World(cave);
        hunter = new Hunter(cave.size(), takeRisks);
        hunter.perceive(world.percept());
        hunter.think();
    }


    /** Окно «Новая пещера». */
    public void newCave(CaveSpec newSpec) {
        spec = Objects.requireNonNull(newSpec, "spec");
        cave = CaveGenerator.generate(newSpec);
        restart();
    }


    /** Кнопка «Пример из задания». */
    public void pdfExample() {
        cave = Cave.pdfExample();
        restart();
    }


    /** Галочка «рисковать»: меняет поведение охотника, поэтому игра начинается сначала. */
    public void setTakeRisks(boolean takeRisks) {
        this.takeRisks = takeRisks;
        restart();
    }


    public boolean isTakeRisks() {
        return takeRisks;
    }


    /** Одно действие охотника. Решение выполнено — сразу думает на новом месте. */
    private void act() {
        world.execute(hunter.next(world.percept()));
        if (world.over()) {
            pause();
            return;
        }
        hunter.perceive(world.percept());
        hunter.think();
    }


    /**
     * Кнопка «Шаг»: одно действие охотника — поворот, шаг вперёд, выстрел... Ставит на
паузу.
     * Новая строка журнала появляется, когда решение выполнено и охотник подумал на новом
месте.
     */
    public void stepOnce() {
        pause();
        if (!world.over()) {
            act();
        }
```

```
    }

    public void start() {
        if (running || world.over()) {
            return;
        }
        running = true;
        hasLastTick = false;
        actionDebt = 0;
    }

    public void pause() {
        running = false;
    }

    /** Одна кнопка «Старт/Пауза». */
    public void toggleRunning() {
        if (running) {
            pause();
        } else {
            start();
        }
    }

    public boolean isRunning() {
        return running;
    }

    /** Очередной кадр окна. nowNanos — текущее время в наносекундах. */
    public void tick(long nowNanos) {
        if (!running) {
            return;
        }
        if (!hasLastTick || nowNanos < lastTick) {
            hasLastTick = true;
            lastTick = nowNanos;
            return;
        }
        // больше секунды за кадр не учитываем: иначе после долгого «замерзания» набежит
гигантский долг
        long elapsed = Math.min(nowNanos - lastTick, NANOS_PER_SECOND);
        lastTick = nowNanos;
        actionDebt += elapsed * speed;
        long due = actionDebt / NANOS_PER_SECOND;
        actionDebt %= NANOS_PER_SECOND;
        for (long i = 0; i < due && running; i++) {
            act();
        }
    }

    /** Скорость в действиях в секунду; значения вне диапазона прижимаются к границе. */
    public void setSpeed(int actionsPerSecond) {
        speed = Math.clamp(actionsPerSecond, MIN_SPEED, MAX_SPEED);
    }

    public int getSpeed() {
        return speed;
    }

    public CaveSpec spec() {
        return spec;
    }

    public Cave cave() {
        return cave;
    }

    public World world() {
```

```
        return world;
    }

    public Hunter hunter() {
        return hunter;
    }
}
```

Файл src/main/java/wumpus/controller/Marks.java

```
package wumpus.controller;

import wumpus.agent.CellKnowledge;
import wumpus.agent.Hunter;
import wumpus.logic.Status;
import wumpus.model.Percept;
import wumpus.model.Position;

import java.util.ArrayList;
import java.util.List;
import java.util.function.Predicate;

/**
 * Пометки на карте охотника, как в таблицах задания: OK, P?, P!, W?, W!, G?, G!.
 * «!» — доказано. «?» ставится только рядом с комнатой, где почувствовали ветерок, запах
или блеск:
 * иначе «может быть» стояло бы почти в каждой неизвестной комнате и карта превратилась бы
в шум.
 */
public final class Marks {

    private Marks() {
    }

    public static List<String> of(Hunter hunter, Position cell) {
        CellKnowledge k = hunter.knowledge().get(cell);
        List<String> result = new ArrayList<>();
        if (k.safe() && !k.visited()) {
            result.add("OK");
        }
        add(result, "P", k.pit(), hunter, cell, Percept::breeze);
        if (!hunter.wumpusDead()) {
            add(result, "W", k.wumpus(), hunter, cell, Percept::stench);
        }
        if (!hunter.hasGold()) {
            add(result, "G", k.gold(), hunter, cell, Percept::glitter);
        }
        return result;
    }

    private static void add(List<String> result, String letter, Status status, Hunter
hunter, Position cell,
                            Predicate<Percept> sensor) {
        if (status == Status.YES) {
            result.add(letter + "!");
        } else if (status == Status.MAYBE && sensedNear(hunter, cell, sensor)) {
            result.add(letter + "?");
        }
    }

    /** Почувствовали ли в соседней посещённой комнате. */
    private static boolean sensedNear(Hunter hunter, Position cell, Predicate<Percept>
sensor) {
        int size = (int) Math.round(Math.sqrt(hunter.knowledge().size()));
        for (Position p : cell.neighbors(size)) {
            Percept seen = hunter.knowledge().get(p).percept();
            if (seen != null && sensor.test(seen)) {
                return true;
            }
```

```
        }
        return false;
    }
}
```

Файл src/main/java/wumpus/controller/FormField.java

```
package wumpus.controller;


import wumpus.util.SafeText;


import java.util.regex.Pattern;


/**
 * Поле формы: ключ (имя параметра), подпись для человека и вид значения.
 * Умеет прочитать текст из поля ввода и превратить значение обратно в текст.
 */
public record FormField(String key, String label, Kind kind) {


    public enum Kind {
        INTEGER,
        LONG
    }


    /** Только ASCII-цифры, необязательный знак: «5» или «٣» числом не считаются. */
    private static final Pattern INTEGER = Pattern.compile("[+-]?[0-9]+");
    private static final int ECHO_LIMIT = 20;


    /** Прочитанное значение или текст ошибки. */
    public sealed interface Read {
        record Value(Object value) implements Read {
        }


        record Error(String message) implements Read {
        }
    }


    public Read read(String raw) {
        String text = raw == null ? "" : raw.strip();
        if (text.isEmpty()) {
            return error("не заполнено");
        }
        String echo = "\"" + SafeText.preview(text, ECHO_LIMIT) + "\"";
        if (!INTEGER.matcher(text).matches()) {
            return error("нужно целое число, а указано " + echo);
        }
        try {
            return new Read.Value(kind == Kind.LONG ? (Object) Long.parseLong(text) :
Integer.parseInt(text));
        } catch (NumberFormatException e) {
            return error("слишком большое число " + echo);
        }
    }


    /** Значение как текст для поля ввода. */
    public static String text(Object value) {
        return String.valueOf(value);
    }


    private Read error(String what) {
        return new Read.Error(label + ": " + what);
    }
}
```

Файл src/main/java/wumpus/controller/FormResult.java

```
package wumpus.controller;
```

```
import wumpus.model.InvalidSettingsException.Problem;

import java.util.List;

/** Результат разбора формы: готовое значение или список понятных проблем. */
public sealed interface FormResult<T> {

    record Valid<T>(T value) implements FormResult<T> {
    }

    record Invalid<T>(List<Problem> problems) implements FormResult<T> {
        public Invalid {
            problems = List.copyOf(problems);
        }
    }
}
```

Файл src/main/java/wumpus/controller/Forms.java

```
package wumpus.controller;

import wumpus.model.InvalidSettingsException;
import wumpus.model.InvalidSettingsException.Problem;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.function.Function;

/** Общая часть форм: сначала читаются все поля, потом проверяются правила самого объекта.
*/
final class Forms {

    private Forms() {
    }

    /**
     * Если хоть одно поле не прочиталось — возвращаются ошибки чтения (по порядку полей).
     * Иначе значения отдаются build, а его {@link InvalidSettingsException} превращается в
список проблем.
     */
    static <T> FormResult<T> parse(List<FormField> fields, Map<String, String> texts,
Function<Object[], T> build) {
        List<Problem> problems = new ArrayList<>();
        Object[] values = new Object[fields.size()];
        for (int i = 0; i < fields.size(); i++) {
            FormField field = fields.get(i);
            switch (field.read(texts.get(field.key()))) {
                case FormField.Read.Value(Object value) -> values[i] = value;
                case FormField.Read.Error(String message) -> problems.add(new
Problem(field.key(), message));
            }
        }
        if (!problems.isEmpty()) {
            return new FormResult.Invalid<>(problems);
        }
        try {
            return new FormResult.Valid<>(build.apply(values));
        } catch (InvalidSettingsException e) {
            return new FormResult.Invalid<>(e.problems());
        }
    }

    static Map<String, String> texts(List<FormField> fields, Object... values) {
        Map<String, String> texts = new LinkedHashMap<>();
```

```
        for (int i = 0; i < fields.size(); i++) {
            texts.put(fields.get(i).key(), FormField.text(values[i]));
        }
        return texts;
    }
}
```

Файл src/main/java/wumpus/controller/CaveForm.java

```
package wumpus.controller;


import wumpus.model.CaveSpec;


import java.util.List;
import java.util.Map;


import static wumpus.controller.FormField.Kind.INTEGER;
import static wumpus.controller.FormField.Kind.LONG;


/** Форма «Новая пещера» без JavaFX: тексты полей → {@link CaveSpec} или список проблем. */
public final class CaveForm {


    /** Поля в том же порядке, что и части {@link CaveSpec}. */
    public static final List<FormField> FIELDS = List.of(
            new FormField("size", "Размер N (пещера N×N)", INTEGER),
            new FormField("pitPercent", "Доля ям, %", INTEGER),
            new FormField("seed", "Зерно случайности (seed)", LONG));


    private CaveForm() {
    }


    public static Map<String, String> toTexts(CaveSpec s) {
        return Forms.texts(FIELDS, s.size(), s.pitPercent(), s.seed());
    }


    public static FormResult<CaveSpec> parse(Map<String, String> texts) {
        return Forms.parse(FIELDS, texts, v -> new CaveSpec((int) v[0], (int) v[1], (long)
v[2]));
    }
}
```

Файл src/main/java/wumpus/controller/SpeedScale.java

```
package wumpus.controller;


/**
 * Перевод положения слайдера скорости в действия в секунду и обратно.
 * Шкала логарифмическая: медленные скорости, удобные для разбора, получают половину
слайдера.
 */
public final class SpeedScale {


    public static final double SLIDER_MIN = 0;
    public static final double SLIDER_MAX = 100;


    private static final double LOG_MIN = Math.log(GameController.MIN_SPEED);
    private static final double LOG_MAX = Math.log(GameController.MAX_SPEED);


    private SpeedScale() {
    }


    /** Положение слайдера → скорость. */
    public static int toSpeed(double slider) {
        double t = (slider - SLIDER_MIN) / (SLIDER_MAX - SLIDER_MIN);
        long speed = Math.round(Math.exp(LOG_MIN + t * (LOG_MAX - LOG_MIN)));
```

```
        // За краями слайдера exp даёт почти 0 или бесконечность, NaN округляется в 0 — всё
прижимается к диапазону
        return (int) Math.clamp(speed, GameController.MIN_SPEED, GameController.MAX_SPEED);
    }


    /** Скорость → положение слайдера. */
    public static double toSlider(int speed) {
        int clamped = Math.clamp(speed, GameController.MIN_SPEED,
GameController.MAX_SPEED);
        return SLIDER_MIN + (Math.log(clamped) - LOG_MIN) / (LOG_MAX - LOG_MIN) *
(SLIDER_MAX - SLIDER_MIN);
    }
}
```

Файл src/main/java/wumpus/controller/Texts.java

```
package wumpus.controller;


import wumpus.agent.Hunter;
import wumpus.model.World;


/** Строки панели состояния. */
public final class Texts {


    private Texts() {
    }


    /** Что охотник знает о Вампусе. */
    public static String wumpus(Hunter h) {
        if (h.wumpusDead()) {
            return "убит";
        }
        return h.knownWumpus() == null ? "не найден" : "в " + h.knownWumpus() + ", жив";
    }


    /** Что охотник знает о золоте. */
    public static String gold(Hunter h) {
        if (h.hasGold()) {
            return "у охотника";
        }
        return h.knownGold() == null ? "не найдено" : "в " + h.knownGold();
    }


    public static String arrow(Hunter h) {
        return h.hasArrow() ? "есть" : "нет";
    }


    /** Итог игры; пока игра идёт — пустая строка. */
    public static String result(World w) {
        if (!w.over()) {
            return "";
        }
        if (w.won()) {
            return "Победа: охотник выбрался с золотом";
        }
        if (!w.alive()) {
            return w.cave().isPit(w.position())
                    ? "Охотник упал в яму в " + w.position()
                    : "Охотника съел Вампус в " + w.position();
        }
        return "Охотник выбрался без золота";
    }


    /** Счёт со знаком: «+974», «−7», «0». */
    public static String score(int v) {
        if (v == 0) {
            return "0";
        }
```

```
        return (v > 0 ? "+" : "-") + Math.abs(v);
    }
}
```

Файл src/main/java/wumpus/util/SafeText.java

```
package wumpus.util;


/** Безопасный показ введённого пользователем текста внутри сообщений. */
public final class SafeText {


    private SafeText() {
    }


    /**
     * Текст для сообщения об ошибке: не длиннее limit символов (дальше «...»),
     * управляющие символы заменены на «?», чтобы они не испортили экран.
     */
    public static String preview(String text, int limit) {
        StringBuilder sb = new StringBuilder();
        text.codePoints()
                .limit(limit)
                .forEach(cp -> sb.appendCodePoint(Character.isISOControl(cp) ? '?' : cp));
        if (text.codePointCount(0, text.length()) > limit) {
            sb.append("...");
        }
        return sb.toString();
    }
}
```

Файл src/main/java/wumpus/view/WumpusApp.java

```
package wumpus.view;


import javafx.animation.AnimationTimer;
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.CheckBox;
import javafx.scene.control.Label;
import javafx.scene.control.ScrollPane;
import javafx.scene.control.Slider;
import javafx.scene.control.TextArea;
import javafx.scene.layout.BorderPane;
import javafx.scene.layout.GridPane;
import javafx.scene.layout.HBox;
import javafx.scene.layout.Priority;
import javafx.scene.layout.Region;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;
import wumpus.agent.Hunter;
import wumpus.agent.Turn;
import wumpus.controller.CaveForm;
import wumpus.controller.GameController;
import wumpus.controller.SpeedScale;
import wumpus.controller.Texts;
import wumpus.model.Cave;
import wumpus.model.CaveSpec;
import wumpus.model.World;


import java.util.List;


/**
 * Главное окно: слева настоящая пещера, справа — что знает охотник, ещё правее состояние,
 * внизу журнал шагов «измерение → вывод → решение». Таймер JavaFX на каждом кадре даёт
контроллеру
 * время, тот решает, сколько действий сделать, а окно перерисовывает изменившееся.
 */
```

```
public class WumpusApp extends Application {

    private final GameController controller = new GameController();
    private final CaveCanvas truth = new CaveCanvas(CaveCanvas.Mode.TRUTH);
    private final CaveCanvas knowledge = new CaveCanvas(CaveCanvas.Mode.KNOWLEDGE);
    private final Label caveInfo = new Label();
    private final Button startPause = new Button();
    private final Button step = new Button("Шаг");
    private final CheckBox showTruth = new CheckBox("показывать правду");
    private final CheckBox takeRisks = new CheckBox("рисковать, когда безопасных ходов
нет");
    private final Slider speed = new Slider(SpeedScale.SLIDER_MIN, SpeedScale.SLIDER_MAX,
0);
    private final Label speedLabel = new Label();
    private final Label stepValue = new Label();
    private final Label actionsValue = new Label();
    private final Label scoreValue = new Label();
    private final Label arrowValue = new Label();
    private final Label wumpusValue = new Label();
    private final Label goldValue = new Label();
    private final Label result = new Label();
    private final TextArea journal = new TextArea();
    private AnimationTimer timer;

    /** Что уже нарисовано: перерисовываем, только когда что-то поменялось. */
    private World drawnWorld;
    private int drawnActions = -1;
    private boolean drawnTruth;
    private int drawnTurns = -1;

    static String stylesheet() {
        return WumpusApp.class.getResource("app.css").toExternalForm();
    }

    @Override
    public void start(Stage stage) {
        BorderPane root = new BorderPane();
        root.setTop(buildToolbar());
        root.setCenter(buildMaps());
        root.setRight(buildPanel());
        root.setBottom(buildJournal());

        Scene scene = new Scene(root, 1180, 800);
        scene.getStylesheets().add(stylesheet());
        stage.setTitle("Мир Вампуса");
        stage.setMinWidth(900);
        stage.setMinHeight(620);
        stage.setScene(scene);

        timer = new AnimationTimer() {
            @Override
            public void handle(long now) {
                controller.tick(now);
                refresh();
            }
        };
        refresh();
        stage.show();
        timer.start();
    }

    @Override
    public void stop() {
        if (timer != null) {
            timer.stop();
        }
    }

    private HBox buildToolbar() {
        Label title = new Label("Мир Вампуса");
```

```
        title.getStyleClass().add("title");
        caveInfo.getStyleClass().add("muted");

        startPause.setId("startPause");
        startPause.setOnAction(e -> {
            controller.toggleRunning();
            refresh();
        });
        step.setId("step");
        step.setOnAction(e -> {
            controller.stepOnce();
            refresh();
        });
        Button restart = new Button("Сначала");
        restart.setId("restart");
        restart.setOnAction(e -> {
            controller.restart();
            refresh();
        });
        Button newCave = new Button("Новая пещера…");
        newCave.setId("newCave");
        newCave.setOnAction(e -> openNewCave());
        Button example = new Button("Пример из задания");
        example.setId("example");
        example.setOnAction(e -> {
            controller.pdfExample();
            refresh();
        });
        for (Button b : List.of(startPause, step, restart, newCave, example)) {
            b.setMinWidth(Region.USE_PREF_SIZE);
        }
        Region spacer = new Region();
        HBox.setHgrow(spacer, Priority.ALWAYS);

        HBox bar = new HBox(8, title, caveInfo, spacer, startPause, step, restart, newCave,
example);
        bar.setAlignment(Pos.CENTER_LEFT);
        bar.getStyleClass().add("toolbar-row");
        return bar;
    }

    private HBox buildMaps() {
        VBox left = map("Пещера (правда)", truth);
        VBox right = map("Что знает охотник", knowledge);
        HBox maps = new HBox(14, left, right);
        HBox.setHgrow(left, Priority.ALWAYS);
        HBox.setHgrow(right, Priority.ALWAYS);
        maps.setPadding(new Insets(10, 12, 6, 12));
        return maps;
    }

    private static VBox map(String caption, CaveCanvas canvas) {
        Label label = new Label(caption);
        label.getStyleClass().add("muted");
        VBox box = new VBox(4, label, canvas);
        VBox.setVgrow(canvas, Priority.ALWAYS);
        box.setMinWidth(160);
        box.setPrefWidth(400);
        return box;
    }

    private ScrollPane buildPanel() {
        Label state = new Label("Состояние");
        state.getStyleClass().add("section");
        GridPane grid = new GridPane();
        grid.setHgap(14);
        grid.setVgap(4);
        String[] names = {"Шаг вывода", "Действий", "Счёт", "Стрела", "Вампус", "Золото"};
        Label[] values = {stepValue, actionsValue, scoreValue, arrowValue, wumpusValue,
goldValue};
        for (int i = 0; i < names.length; i++) {
            Label name = new Label(names[i]);
```

```
            name.getStyleClass().add("muted");
            values[i].getStyleClass().add("value");
            grid.add(name, 0, i);
            grid.add(values[i], 1, i);
        }
        result.getStyleClass().add("status");
        result.setWrapText(true);


        showTruth.setId("showTruth");
        showTruth.setSelected(true);
        showTruth.setOnAction(e -> refresh());
        takeRisks.setId("takeRisks");
        takeRisks.setSelected(controller.isTakeRisks());
        takeRisks.setWrapText(true);
        takeRisks.setOnAction(e -> {
            controller.setTakeRisks(takeRisks.isSelected());
            refresh();
        });
        speed.setValue(SpeedScale.toSlider(controller.getSpeed()));
        speed.valueProperty().addListener((o, a, b) -> {
            controller.setSpeed(SpeedScale.toSpeed(b.doubleValue()));
            refresh();
        });


        Label legendTitle = new Label("Обозначения");
        legendTitle.getStyleClass().add("section");
        Label legend = new Label("""
                серая клетка — охотник там был
                OK — доказано, что безопасно
                P? / W? / G? — может быть яма / Вампус / золото
                P! / W! / G! — доказано
                волны — запах, линии — ветерок, звезда — блеск""");
        legend.getStyleClass().add("legend");
        legend.setWrapText(true);


        Label settings = new Label("Настройки");
        settings.getStyleClass().add("section");
        VBox panel = new VBox(6, state, grid, result, settings, showTruth, takeRisks,
                new Label("Скорость"), speed, speedLabel, legendTitle, legend);
        panel.setPadding(new Insets(12));
        // в низком окне панель прокручивается, а не обрезается
        ScrollPane scroll = new ScrollPane(panel);
        scroll.setFitToWidth(true);
        scroll.setHbarPolicy(ScrollPane.ScrollBarPolicy.NEVER);
        scroll.getStyleClass().add("panel");
        scroll.setPrefWidth(290);
        scroll.setMinWidth(250);
        return scroll;
    }


    private VBox buildJournal() {
        Label caption = new Label("Журнал: измерение → вывод → решение");
        caption.getStyleClass().add("muted");
        journal.setEditable(false);
        journal.setWrapText(true);
        journal.setPrefRowCount(7);
        journal.setId("journal");
        VBox box = new VBox(4, caption, journal);
        box.setPadding(new Insets(6, 12, 10, 12));
        return box;
    }


    private void openNewCave() {
        controller.pause();
        new FormDialog<>("Новая пещера",
                "Расстановка случайная по seed. Ям — указанная доля от числа комнат.\nСтарт
и соседние с ним комнаты всегда без ям и Вампуса.",
                "Создать", CaveForm.FIELDS, CaveForm.toTexts(controller.spec()),
CaveForm.toTexts(CaveSpec.defaults()),
                CaveForm::parse)
                .showAndWait().ifPresent(controller::newCave);
        refresh();
```

```
    }

    private void refresh() {
        World world = controller.world();
        Hunter hunter = controller.hunter();
        Cave cave = controller.cave();
        caveInfo.setText("пещера " + cave.size() + "×" + cave.size() + ", ям " +
cave.pits().size());
        startPause.setText(controller.isRunning() ? "Пауза" : "Старт");
        startPause.setDisable(world.over());
        step.setDisable(world.over());
        speedLabel.setText(controller.getSpeed() + " действий в секунду");
        speedLabel.getStyleClass().setAll("label", "muted");

        List<Turn> turns = hunter.journal();
        boolean changed = world != drawnWorld || world.actions() != drawnActions
                || showTruth.isSelected() != drawnTruth || turns.size() != drawnTurns;
        if (!changed) {
            return;
        }
        drawnWorld = world;
        drawnActions = world.actions();
        drawnTruth = showTruth.isSelected();

        truth.show(world, hunter, !showTruth.isSelected());
        knowledge.show(world, hunter, false);
        stepValue.setText(Integer.toString(turns.size()));
        actionsValue.setText(Integer.toString(world.actions()));
        scoreValue.setText(Texts.score(world.score()));
        arrowValue.setText(Texts.arrow(hunter));
        wumpusValue.setText(Texts.wumpus(hunter));
        goldValue.setText(Texts.gold(hunter));
        result.setText(Texts.result(world));
        result.getStyleClass().setAll("label", world.won() || !world.over() ? "status" :
"message");

        if (turns.size() != drawnTurns) {
            StringBuilder text = new StringBuilder();
            for (Turn t : turns) {
                text.append(t).append('\n');
            }
            journal.setText(text.toString().strip());
            journal.positionCaret(journal.getLength());
            journal.setScrollTop(Double.MAX_VALUE);
            drawnTurns = turns.size();
        }
    }
}
```

Файл src/main/java/wumpus/view/CaveCanvas.java

```
package wumpus.view;

import javafx.geometry.VPos;
import javafx.scene.canvas.Canvas;
import javafx.scene.canvas.GraphicsContext;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import javafx.scene.text.TextAlignment;
import wumpus.agent.Hunter;
import wumpus.controller.Marks;
import wumpus.model.Cave;
import wumpus.model.Percept;
import wumpus.model.Position;
import wumpus.model.World;

import java.util.List;
```

```
/**
 * Карта пещеры: либо настоящая (что где лежит и что чувствуется), либо то, что знает
охотник.
 * Растягивается под размер окна, клетки квадратные. В углу клетки — её координаты, как в
таблицах задания.
 */
public class CaveCanvas extends Pane {


    public enum Mode {
        TRUTH,
        KNOWLEDGE
    }


    static final Color GRID = Color.web("#d3d1c7");
    static final Color CELL = Color.web("#ffffff");
    static final Color VISITED = Color.web("#e4e2dc");
    static final Color START = Color.web("#e6f1fb");
    static final Color MUTED = Color.web("#888780");
    static final Color OK = Color.web("#0f6e56");
    static final Color MAYBE = Color.web("#854f0b");
    static final Color SURE = Color.web("#a32d2d");
    static final Color HIDDEN = Color.web("#f1efe8");


    private static final double MARGIN = 4;
    /** В клетках мельче координаты налезают на значки — тогда их не пишем. */
    private static final double MIN_CELL_FOR_LABEL = 36;


    private final Mode mode;
    private final Canvas canvas = new Canvas();
    private World world;
    private Hunter hunter;
    private boolean hidden;


    public CaveCanvas(Mode mode) {
        this.mode = mode;
        getChildren().add(canvas);
        setMinSize(160, 160);
        widthProperty().addListener((o, a, b) -> redraw());
        heightProperty().addListener((o, a, b) -> redraw());
    }


    public void show(World world, Hunter hunter, boolean hidden) {
        this.world = world;
        this.hunter = hunter;
        this.hidden = hidden;
        redraw();
    }


    @Override
    protected void layoutChildren() {
        canvas.setWidth(getWidth());
        canvas.setHeight(getHeight());
    }


    private void redraw() {
        canvas.setWidth(getWidth());
        canvas.setHeight(getHeight());
        GraphicsContext g = canvas.getGraphicsContext2D();
        g.clearRect(0, 0, canvas.getWidth(), canvas.getHeight());
        if (world == null) {
            return;
        }
        Cave cave = world.cave();
        int n = cave.size();
        double cell = Math.floor(Math.min(getWidth() - 2 * MARGIN, getHeight() - 2 *
MARGIN) / n);
        if (cell <= 4) {
            return;
        }
        double left = Math.floor((getWidth() - cell * n) / 2);
```

```
        double top = MARGIN;
        if (hidden) {
            g.setFill(HIDDEN);
            g.fillRoundRect(left, top, cell * n, cell * n, 8, 8);
            g.setFill(MUTED);
            g.setFont(Font.font("PT Sans", 14));
            g.setTextAlign(TextAlignment.CENTER);
            g.setTextBaseline(VPos.CENTER);
            g.fillText("Пещера скрыта —\nохотник её тоже не видит", left + cell * n / 2,
top + cell * n / 2);
            return;
        }
        for (int row = 1; row <= n; row++) {
            for (int col = 1; col <= n; col++) {
                Position p = new Position(row, col);
                double x = left + (col - 1) * cell;
                double y = top + (row - 1) * cell;
                if (mode == Mode.TRUTH) {
                    drawTruth(g, p, x, y, cell);
                } else {
                    drawKnowledge(g, p, x, y, cell);
                }
                g.setStroke(GRID);
                g.setLineWidth(1);
                g.strokeRect(x + 0.5, y + 0.5, cell - 1, cell - 1);
                if (cell < MIN_CELL_FOR_LABEL) {
                    continue;
                }
                g.setFill(MUTED);
                g.setFont(Font.font("PT Sans", Math.max(8, cell * 0.14)));
                g.setTextAlign(TextAlignment.LEFT);
                g.setTextBaseline(VPos.TOP);
                g.fillText(p.toString(), x + cell * 0.06, y + cell * 0.04);
            }
        }
        Position h = world.position();
        double hx = left + (h.col() - 0.5) * cell;
        double hy = top + (h.row() - 0.5) * cell;
        Icons.hunter(g, hx, hy, cell, world.facing());
    }


    private void drawTruth(GraphicsContext g, Position p, double x, double y, double cell)
{
        Cave cave = world.cave();
        g.setFill(p.equals(cave.start()) ? START : CELL);
        g.fillRect(x, y, cell, cell);
        double cx = x + cell / 2;
        double cy = y + cell / 2;
        if (cave.isPit(p)) {
            Icons.pit(g, cx, cy, cell, Icons.PIT);
        }
        if (cave.wumpus().equals(p)) {
            Icons.wumpus(g, cx, cy, cell, world.wumpusAlive());
        }
        if (cave.gold().equals(p) && !world.hasGold()) {
            Icons.gold(g, cx, cy, cell);
        }
        sensors(g, x, y, cell, cave.hasStench(p), cave.hasBreeze(p), cave.nearGold(p) &&
!world.hasGold());
    }


    private void drawKnowledge(GraphicsContext g, Position p, double x, double y, double
cell) {
        // измерения — свежие, сразу после шага; пометки — с последнего вывода
        Percept seen = hunter.perceptAt(p);
        g.setFill(seen != null ? VISITED : CELL);
        g.fillRect(x, y, cell, cell);
        if (seen != null) {
            sensors(g, x, y, cell, seen.stench(), seen.breeze(), seen.glitter());
        }
        List<String> marks = Marks.of(hunter, p);
        if (marks.isEmpty() || p.equals(world.position())) {
            return;
        }
```

```
        double fontSize = Math.max(9, cell * (marks.size() > 1 ? 0.2 : 0.26));
        g.setFont(Font.font("PT Sans", FontWeight.BOLD, fontSize));
        g.setTextAlign(TextAlignment.CENTER);
        g.setTextBaseline(VPos.CENTER);
        double lineHeight = fontSize * 1.1;
        double start = y + cell / 2 - (marks.size() - 1) * lineHeight / 2;
        for (int i = 0; i < marks.size(); i++) {
            String m = marks.get(i);
            g.setFill(m.equals("OK") ? OK : m.endsWith("?") ? MAYBE : m.startsWith("G") ?
Icons.GOLD_EDGE : SURE);
            g.fillText(m, x + cell / 2, start + i * lineHeight);
        }
    }


    /** Значки датчиков в нижней полосе клетки. */
    private static void sensors(GraphicsContext g, double x, double y, double cell, boolean
stench, boolean breeze, boolean glitter) {
        double s = cell * 0.2;
        double sx = x + cell * 0.08;
        double sy = y + cell - s - cell * 0.06;
        if (stench) {
            Icons.stench(g, sx, sy, s);
            sx += s * 1.3;
        }
        if (breeze) {
            Icons.breeze(g, sx, sy, s);
            sx += s * 1.3;
        }
        if (glitter) {
            Icons.glitter(g, sx, sy, s);
        }
    }
}
```

Файл src/main/java/wumpus/view/Icons.java

```
package wumpus.view;


import javafx.scene.canvas.GraphicsContext;
import javafx.scene.paint.Color;
import javafx.scene.shape.ArcType;
import javafx.scene.shape.StrokeLineCap;
import wumpus.model.Direction;


/** Значки, нарисованные фигурами. (cx, cy) — центр, size — размер клетки. */
final class Icons {


    static final Color HUNTER = Color.web("#185fa5");
    static final Color ARROW = Color.web("#e24b4a");
    static final Color WUMPUS = Color.web("#a32d2d");
    static final Color DEAD = Color.web("#b4b2a9");
    static final Color PIT = Color.web("#2c2c2a");
    static final Color GOLD = Color.web("#ef9f27");
    static final Color GOLD_EDGE = Color.web("#ba7517");
    static final Color STENCH = Color.web("#3b6d11");
    static final Color BREEZE = Color.web("#888780");


    private Icons() {
    }


    /** Охотник: голова и плечи; красная стрелка у края клетки — куда он смотрит (как на
картинке задания). */
    static void hunter(GraphicsContext g, double cx, double cy, double size, Direction
facing) {
        double r = size * 0.1;
        g.setFill(HUNTER);
        g.fillOval(cx - r, cy - size * 0.16 - r, 2 * r, 2 * r);
        g.fillArc(cx - size * 0.17, cy - size * 0.03, size * 0.34, size * 0.3, 0, 180,
ArcType.ROUND);
        double from = size * 0.24;
```

```
        double tip = size * 0.44;
        double head = size * 0.08;
        double sx = cx + facing.dCol() * from;
        double sy = cy + facing.dRow() * from;
        double px = cx + facing.dCol() * tip;
        double py = cy + facing.dRow() * tip;
        double bx = px - facing.dCol() * head * 1.4;
        double by = py - facing.dRow() * head * 1.4;
        double ox = facing.dRow() * head;
        double oy = facing.dCol() * head;
        g.setStroke(ARROW);
        g.setLineWidth(Math.max(1.5, size * 0.035));
        g.setLineCap(StrokeLineCap.ROUND);
        g.strokeLine(sx, sy, bx, by);
        g.setFill(ARROW);
        g.fillPolygon(new double[]{px, bx + ox, bx - ox}, new double[]{py, by + oy, by -
oy}, 3);
    }

    /** Вампус: круглое тело, глаза и зубы. */
    static void wumpus(GraphicsContext g, double cx, double cy, double size, boolean alive)
{
        double r = size * 0.28;
        g.setFill(alive ? WUMPUS : DEAD);
        g.fillOval(cx - r, cy - r, 2 * r, 2 * r);
        g.setFill(Color.WHITE);
        double e = size * 0.06;
        g.fillOval(cx - size * 0.12 - e, cy - size * 0.08 - e, 2 * e, 2 * e);
        g.fillOval(cx + size * 0.12 - e, cy - size * 0.08 - e, 2 * e, 2 * e);
        double ty = cy + size * 0.1;
        for (int i = -1; i <= 1; i++) {
            double tx = cx + i * size * 0.08;
            g.fillPolygon(new double[]{tx - size * 0.035, tx + size * 0.035, tx},
                    new double[]{ty, ty, ty + size * 0.07}, 3);
        }
        if (!alive) {
            g.setStroke(Color.WHITE);
            g.setLineWidth(Math.max(1, size * 0.03));
            for (int s : new int[]{-1, 1}) {
                double ex = cx + s * size * 0.12;
                double ey = cy - size * 0.08;
                g.strokeLine(ex - e, ey - e, ex + e, ey + e);
                g.strokeLine(ex - e, ey + e, ex + e, ey - e);
            }
        }
    }

    static void pit(GraphicsContext g, double cx, double cy, double size, Color color) {
        double rx = size * 0.32;
        double ry = size * 0.26;
        g.setFill(color);
        g.fillOval(cx - rx, cy - ry, 2 * rx, 2 * ry);
    }

    static void gold(GraphicsContext g, double cx, double cy, double size) {
        double r = size * 0.2;
        g.setFill(GOLD);
        g.fillOval(cx - r, cy - r, 2 * r, 2 * r);
        g.setStroke(GOLD_EDGE);
        g.setLineWidth(Math.max(1, size * 0.03));
        g.strokeOval(cx - r * 0.6, cy - r * 0.6, 1.2 * r, 1.2 * r);
    }

    /** Запах: три волнистые линии вверх. */
    static void stench(GraphicsContext g, double x, double y, double s) {
        g.setStroke(STENCH);
        g.setLineWidth(Math.max(1, s * 0.09));
        g.setLineCap(StrokeLineCap.ROUND);
        for (int i = 0; i < 3; i++) {
            double lx = x + s * (0.2 + 0.3 * i);
            g.beginPath();
            g.moveTo(lx, y + s);
```

```
            g.bezierCurveTo(lx - s * 0.15, y + s * 0.66, lx + s * 0.15, y + s * 0.33, lx,
y);
            g.stroke();
        }
    }

    /** Ветерок: две изогнутые линии. */
    static void breeze(GraphicsContext g, double x, double y, double s) {
        g.setStroke(BREEZE);
        g.setLineWidth(Math.max(1, s * 0.09));
        g.setLineCap(StrokeLineCap.ROUND);
        for (int i = 0; i < 2; i++) {
            double ly = y + s * (0.3 + 0.4 * i);
            g.beginPath();
            g.moveTo(x, ly);
            g.bezierCurveTo(x + s * 0.4, ly - s * 0.2, x + s * 0.6, ly + s * 0.2, x + s, ly
- s * 0.1);
            g.stroke();
        }
    }

    /** Блеск: четырёхлучевая звёздочка. */
    static void glitter(GraphicsContext g, double x, double y, double s) {
        double cx = x + s / 2;
        double cy = y + s / 2;
        double a = s / 2;
        double b = s * 0.12;
        g.setFill(GOLD);
        g.fillPolygon(new double[]{cx, cx + b, cx + a, cx + b, cx, cx - b, cx - a, cx - b},
                new double[]{cy - a, cy - b, cy, cy + b, cy + a, cy + b, cy, cy - b}, 8);
    }
}
```

Файл src/main/java/wumpus/view/FormDialog.java

```
package wumpus.view;

import javafx.event.ActionEvent;
import javafx.geometry.Insets;
import javafx.scene.control.Button;
import javafx.scene.control.ButtonBar;
import javafx.scene.control.ButtonType;
import javafx.scene.control.Dialog;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.layout.ColumnConstraints;
import javafx.scene.layout.GridPane;
import javafx.scene.layout.HBox;
import javafx.scene.layout.Region;
import javafx.scene.layout.VBox;
import wumpus.controller.FormField;
import wumpus.controller.FormResult;
import wumpus.model.InvalidSettingsException.Problem;

import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Random;
import java.util.function.Function;

/**
 * Окно с формой. Проверка введённого — в контроллере (CaveForm);
 * здесь только поля, подсветка ошибок и кнопки. С ошибками окно не закрывается.
 */
class FormDialog<T> extends Dialog<T> {

    private final Map<String, TextField> inputs = new LinkedHashMap<>();
    private final Label errors = new Label();
    private final Function<Map<String, String>, FormResult<T>> parser;
```

```
    private T accepted;

    /** @param defaults тексты для кнопки «По умолчанию» */
    FormDialog(String title, String header, String applyText, List<FormField> fields,
               Map<String, String> current, Map<String, String> defaults,
               Function<Map<String, String>, FormResult<T>> parser) {
        this.parser = parser;
        setTitle(title);
        setHeaderText(header);

        GridPane grid = new GridPane();
        grid.setHgap(10);
        grid.setVgap(6);
        grid.setPadding(new Insets(4, 4, 4, 4));
        // подписи никогда не обрезаются многоточием: столбец не уже самой длинной подписи
        ColumnConstraints labelColumn = new ColumnConstraints();
        labelColumn.setMinWidth(Region.USE_PREF_SIZE);
        grid.getColumnConstraints().addAll(labelColumn, new ColumnConstraints(), new
ColumnConstraints());
        int row = 0;
        for (FormField field : fields) {
            TextField input = new TextField(current.get(field.key()));
            input.setPrefColumnCount(10);
            input.setId("field-" + field.key());
            inputs.put(field.key(), input);
            grid.add(new Label(field.label()), 0, row);
            grid.add(input, 1, row);
            if (field.key().equals("seed")) {
                Button random = new Button("Случайный");
                random.setMinWidth(Region.USE_PREF_SIZE);
                random.setOnAction(e -> input.setText(Long.toString(new
Random().nextLong())));
                grid.add(random, 2, row);
            }
            row++;
        }

        Button reset = new Button("По умолчанию");
        reset.setOnAction(e -> defaults.forEach((key, text) ->
inputs.get(key).setText(text)));

        errors.getStyleClass().add("error-text");
        errors.setWrapText(true);
        errors.setMaxWidth(520);
        // блок ошибок растёт по высоте под все строки, а не обрезается
        errors.setMinHeight(Region.USE_PREF_SIZE);

        getDialogPane().setContent(new VBox(10, grid, new HBox(reset), errors));
        ButtonType apply = new ButtonType(applyText, ButtonBar.ButtonData.OK_DONE);
        ButtonType cancel = new ButtonType("Отмена", ButtonBar.ButtonData.CANCEL_CLOSE);
        getDialogPane().getButtonTypes().addAll(apply, cancel);
        getDialogPane().getStylesheets().add(WumpusApp.stylesheet());

        Button applyButton = (Button) getDialogPane().lookupButton(apply);
        applyButton.setId("apply");
        applyButton.addEventFilter(ActionEvent.ACTION, event -> {
            if (!tryAccept()) {
                event.consume(); // есть ошибки — окно остаётся открытым
            }
        });
        setResultConverter(button -> button == apply ? accepted : null);
    }

    private boolean tryAccept() {
        Map<String, String> texts = new HashMap<>();
        inputs.forEach((key, input) -> texts.put(key, input.getText()));
        inputs.values().forEach(input -> input.getStyleClass().remove("invalid"));
        switch (parser.apply(texts)) {
            case FormResult.Valid<T>(T value) -> {
                accepted = value;
```

```
                return true;
            }
            case FormResult.Invalid<T>(List<Problem> problems) -> {
                StringBuilder text = new StringBuilder();
                for (Problem problem : problems) {
                    text.append("• ").append(problem.message()).append('\n');
                    TextField input = inputs.get(problem.field());
                    if (input != null && !input.getStyleClass().contains("invalid")) {
                        input.getStyleClass().add("invalid");
                    }
                }
                errors.setText(text.toString().strip());
                return false;
            }
        }
    }
}
```

Файл src/main/resources/wumpus/view/app.css

```
/* Светлый плоский стиль: белые поверхности, тонкие границы, без теней и градиентов */
.root {
    /* PT Sans — шрифт, созданный под кириллицу: системный шрифт Mac в JavaFX рвёт буквы,
жирный Arial тяжёлый.
       Если шрифта нет (не macOS), JavaFX возьмёт шрифт по умолчанию. */
    -fx-font-family: "PT Sans";
    -fx-font-size: 13px;
    -fx-background-color: #ffffff;
    -fx-base: #f7f6f3;
    -fx-accent: #185fa5;
    -fx-focus-color: #378add;
    -fx-faint-focus-color: transparent;
}


.toolbar-row {
    -fx-padding: 8 12 8 12;
    -fx-border-color: #e4e2dc;
    -fx-border-width: 0 0 1 0;
}


.title {
    -fx-font-size: 15px;
    -fx-font-weight: bold;
}


.muted {
    -fx-text-fill: #6b6a65;
}


.panel {
    -fx-padding: 12;
    -fx-border-color: #e4e2dc;
    -fx-border-width: 0 0 0 1;
}


.legend {
    -fx-text-fill: #6b6a65;
}


.legend .label {
    -fx-text-fill: #6b6a65;
    -fx-font-size: 12px;
}


.button, .toggle-button, .text-field {
    -fx-background-radius: 6;
}


.stat {
```

```
    -fx-background-color: #f1efe8;
    -fx-background-radius: 8;
    -fx-padding: 6 10 6 10;
}

.stat-name {
    -fx-text-fill: #6b6a65;
    -fx-font-size: 11px;
}

.stat-value {
    -fx-font-size: 17px;
    -fx-font-weight: bold;
}

.status {
    -fx-text-fill: #0f6e56;
    -fx-font-weight: bold;
}

.message {
    -fx-text-fill: #854f0b;
}

.section {
    -fx-font-weight: bold;
    -fx-padding: 8 0 0 0;
}

.warning-text {
    -fx-text-fill: #854f0b;
}

.error-text {
    -fx-text-fill: #a32d2d;
}

.text-field.invalid {
    -fx-border-color: #e24b4a;
    -fx-border-width: 2;
    -fx-border-radius: 6;
}

.value {
    -fx-font-weight: bold;
}

.text-area {
    -fx-font-size: 12.5px;
}

.scroll-pane.panel, .scroll-pane.panel > .viewport {
    -fx-background-color: #ffffff;
    -fx-background-insets: 0;
}

.scroll-pane.panel {
    -fx-padding: 0;
    -fx-border-color: #e4e2dc;
    -fx-border-width: 0 0 0 1;
}
```